

Valuation of Financial Derivatives under X-Value Adjustments using Deep Learning Techniques

Lindani Melusi Hlophe

Student Number: 2579491



July 12, 2026

*MSc in Computational and Applied Mathematics,
University of the Witwatersrand.*

A dissertation submitted to the Faculty of Science, University of the Witwatersrand, Johannesburg, in partial fulfilment of the requirements for the degree of Master of Science.

Abstract

This study develops deep learning surrogate models for the direct computation of Credit Valuation Adjustment (CVA) and Debit Valuation Adjustment (DVA) for an interest rate swap (IRS) under the Hull-White one-factor model. Standard xVA computation requires a nested Monte Carlo simulation in which the outer loop evolves the short rate along simulated paths and the inner loop re-prices the derivative at each monitoring date and scenario. This nested structure grows rapidly in computational cost with the number of scenarios and monitoring dates, and hence creates a computational bottleneck. We address this by training two differential deep learning surrogates to directly approximate the Expected Positive Exposure (EPE) and Expected Negative Exposure (ENE) profiles as explicit functions of the initial yield curve, Hull-White parameters, and monitoring time. Since EPE and ENE are already expectations over the short rate distribution, the short rate itself does not appear as a surrogate input, and CVA and DVA are recovered at inference time through a single forward pass per monitoring date with no simulation. Each surrogate is trained jointly on exposure values and their analytic curve sensitivities, which regularizes the learned function and improves accuracy under yield curve shapes not well represented in the training distribution. The trained surrogates reproduce EPE, ENE, CVA, and DVA estimates consistent with a full Monte Carlo benchmark across a range of yield-curve environments, whilst eliminating simulation cost during inference. These results establish that the deep learning surrogates offer a practically viable approach to xVA computation, and the framework is sufficiently general to extend to instruments with more complex payoff structures.

List of Figures

5.1 Typical yield curve shapes: upward-sloping (normal), flat, and inverted. .	41
7.1 A feedforward MLP with $L = 3$ layers. Each node applies an activation function to a weighted sum of the preceding layer's outputs. Weight matrices $W_k \in \mathbb{R}^{d_{k-1} \times d_k}$ connect adjacent layers; the layer widths $d_0, \dots, d_L$ are illustrative.	56
7.2 The RNN unrolled across three time steps. The recurrence $\mathbf{h}_t = g(W_x \mathbf{x}_t + W_h \mathbf{h}_{t-1} + \mathbf{b})$ applies the same parameter matrices at every step. Training by BPTT requires propagating gradients through the full unrolled graph; for a dependency spanning k steps, this involves a product of k Jacobians that decays or explodes exponentially.	61
7.3 Internal dataflow of an LSTM cell at time step t . The cell state $\mathbf{c}_t$ (horizontal channel) carries long-range memory; the forget gate $\mathbf{f}_t$, input gate $\mathbf{i}_t$, and output gate $\mathbf{o}_t$ regulate what is erased, written, and read at each step. The hidden state $\mathbf{h}_t$ is derived from the cell state via the output gate.	63
7.4 Internal dataflow of a GRU cell at time step t . The update gate $\mathbf{z}_t$ interpolates between the previous state $\mathbf{h}_{t-1}$ (weighted by $1 - \mathbf{z}_t$) and the candidate $\tilde{\mathbf{h}}_t$ (weighted by $\mathbf{z}_t$). The reset gate $\mathbf{r}_t$ controls how much of $\mathbf{h}_{t-1}$ enters the candidate computation. The near-direct path from $\mathbf{h}_{t-1}$ to $\mathbf{h}_t$ when $\mathbf{z}_t \approx 0$ is the mechanism that prevents gradient vanishing across long sequences. .	65
8.1 Svensson yield curves (left) and corresponding discount factor curves (right) for five randomly sampled historical dates from the Federal Reserve GSW dataset. The selection illustrates the range of term structure shapes used as proxies for market yield curve data in this study.	73
8.2 Simulated Hull-White short-rate paths over the ten-year IRS horizon, initialised at $r_0 = \beta_0 + \beta_1$ from the Svensson term structure. The upward drift of the path ensemble reflects the positive slope of the initial forward curve embedded in $\theta(t)$, consistent with an upward-sloping initial yield curve.	74

List of Tables

5.1 Interpolation Methods	39
8.1 Confirmation of the interest rate swap transaction, presented in the format of a standard ISDA confirmation	71

List of Notations

Notation

Contents

Abstract	1
List of Figures	1
List of Tables	2
List of Notations	3
Notation	4
I INTRODUCTION AND BACKGROUND	6
1 Introduction	7
2 Background and Literature Review	12
3 Thesis Outline	20
II THEORETICAL FOUNDATIONS	21
4 Valuation of Financial Derivatives	22
5 Interest Rate Curves and Term Structure Models	30
6 Counterparty Credit Risk and xVAs	49
7 Neural Networks and Deep Learning	54

III	METHODOLOGY AND MODELS	69
8	Methodology	70
8.1	Introduction	70
8.2	Model Assumptions	72
8.3	Numerical Pricing of the IRS Under Hull-White	75
8.4	Monte Carlo Simulation and Exposure	77
8.5	Surrogate Models	78
8.5.1	Data Generation	78
8.5.2	Model Inputs	78
8.5.3	Architecture	79
8.5.4	Training Objectives	79
8.6	xVA Calculation	80
IV	RESULTS AND ANALYSIS	81
V	CONCLUSION AND FUTURE WORK	82
VI	APPENDICES	83
A	Mathematical Proofs and Derivations	84
B	Code Implementation Details	85
C	Additional Numerical Results	86
	REFERENCES	93

Part I

Introduction and Background

Chapter 1

Introduction

Financial derivatives are contracts whose payoff depends on the value of an underlying asset, such as an interest rate, an exchange rate, an equity price, or a commodity price [13]. There are many different types of financial derivatives contracts, such as options, forwards, futures and swaps. They are used extensively by financial institutions, corporations, and governments for hedging exposures, managing risk, and taking speculative positions [57]. The accurate valuation of these instruments is therefore not merely a technical exercise but a practical necessity with direct implications for risk management, regulatory compliance, and financial stability [8].

The standard approach to pricing financial derivatives rests on the principle of risk-neutral valuation, which states that the fair value of a derivative is the expected value of its future payoff, discounted at the risk-free rate, under a specially constructed probability measure [12, 25]. This measure is derived from the requirement that no riskless profit can be extracted from the market [47, 48]. The elegance of this framework lies in its generality, once the risk-neutral measure is established, the price of any derivative reduces to computing an expectation, regardless of the specific form of the payoff.

In practice, however, the classical risk-neutral framework rests on assumptions that are difficult to defend in real financial markets. It assumes that all market participants can borrow and lend freely at a common risk-free rate, that there are no costs associated with collateral or funding, and that the risk of counterparty default is negligible [42]. The global financial crisis of 2007-2009 made clear that these were not merely simplifications but material omissions. The default of Lehman Brothers demonstrated that counterparty credit risk was real, that funding costs varied significantly across institutions, and that the assumption of a universal risk-free rate was an idealisation the market could no longer sustain [42, 41].

In response, the financial industry adopted a framework in which the economic value of a derivative is computed as the classical risk-free price adjusted by a set of additive corrections, collectively referred to as x-value adjustments (XVAs) [72]. Each adjustment captures a specific cost or risk that the classical framework had ignored. Credit Valuation Adjustment (CVA) accounts for the expected loss arising from counterparty default. Debit Valuation Adjustment (DVA) reflects the institution's own default risk as seen by its counterparty. Funding Valuation Adjustment (FVA) captures the cost of funding uncollateralised exposures [42]. The framework has since been extended to include Margin Valuation Adjustment (MVA) for the cost of posting initial margin and Capital Valuation Adjustment (KVA) for the ongoing cost of holding regulatory capital [41]. Together, these adjustments bring the theoretical price of a derivative closer to its true economic cost.

Central to the computation of most XVA components is the concept of credit exposure¹. Exposure measures the potential loss to a party in the event that its counterparty defaults at some future point in time [42]. It is not a fixed quantity, as market conditions evolve, so does the value of the derivative and hence the potential loss. XVA calculations therefore require a complete picture of how a derivative's value may evolve over its entire lifetime under a wide range of market scenarios. This picture is captured by the exposure profile, specifically the expected positive exposure (EPE) and expected negative exposure (ENE), which average the positive and negative values of the derivative across simulated scenarios at each monitoring date². CVA and DVA are computed by integrating these profiles against default probabilities and survival functions, making accurate exposure profile estimation the central computational task in XVA valuation [42].

Computing exposure profiles under classical methods requires simulating thousands of future market scenarios and re-pricing the derivative at each monitoring date along every simulated path [30]. This produces a nested simulation structure: an outer loop over market scenarios and an inner loop of re-pricing evaluations at each time step. The computational cost scales as the product of the number of outer paths and the cost of the inner re-pricing, making it expensive even for simple instruments [72]. For derivatives with complex payoff structures, path-dependent features, or long

¹Credit exposure should not be confused with Value-at-Risk (VaR)

²A monitoring date is a discrete future time point at which exposure is evaluated

maturities, the cost becomes prohibitive. Large financial institutions manage portfolios of thousands of trades, each requiring its own exposure simulation, and must perform these calculations frequently for risk reporting and regulatory purposes [61]. Classical Monte Carlo methods are increasingly inadequate to meet this demand within the time constraints of real-world operations [30].

Deep learning surrogate models offer a practical approach to this problem. A surrogate is a neural network trained to approximate a function that is expensive to evaluate directly. In our context, the surrogate models will target the computation of the exposure profiles. Rather than simulating paths and re-pricing at each node, this study trains two deep learning surrogates to approximate the EPE and ENE profiles directly as functions of the initial yield curve, Hull-White model parameters, and monitoring time. Once trained, the surrogate can generate exposure profiles at a fraction of the cost of full Monte Carlo simulation. This technique is similar to [35], in which

Machine learning and deep learning have found broad application in quantitative finance as tools for overcoming the computational limitations of classical numerical methods. The core appeal is their capacity to approximate complex, high-dimensional functions at negligible inference cost once training is complete [38]. In derivative pricing, this has proven particularly valuable for problems governed by high-dimensional partial differential equations, where classical finite difference and Monte Carlo methods suffer from an exponential growth in computational cost with the number of state variables [46, 9]. The recognition that neural networks can serve as fast, accurate function approximators for such problems has driven a substantial body of research at the intersection of machine learning and computational finance [75].

In the specific context of xVA computation, surrogate models have emerged as a practical solution to the nested simulation problem. Several studies have demonstrated that neural networks can be trained to approximate the mark-to-market value of an interest rate derivative across a range of market states, replacing the expensive inner re-pricing at each scenario node with a single forward pass [35, 64]. This approach has been extended to high-dimensional Bermudan options [3], to bilateral counterparty risk under stochastic intensity models [37], and to the computation of exposure and xVA profiles for portfolios of trades [78]. The consistent finding across this literature is that well-designed surrogate architectures reproduce exposure profiles with high accuracy whilst reducing computation time substantially relative to full Monte Carlo simulation [30].

A limitation of standard surrogate training is that optimising on price or exposure labels alone does not guarantee that the network learns accurate sensitivities. In xVA applications, curve sensitivities are indispensable, they feed directly into hedging decisions and regulatory capital calculations, and a surrogate that prices correctly but produces incorrect gradients is of limited practical value [30]. Differential deep learning addresses this by incorporating analytic sensitivities as auxiliary training targets, optimizing the network jointly on values and their derivatives with respect to the input yield curve. This additional gradient information regularizes the learned mapping, improves generalization to sparsely sampled regions of the input space, and produces sensitivity estimates consistent with the underlying financial model [63].

This study applies deep learning to approximate the Expected Positive Exposure and Expected Negative Exposure profiles of an interest rate swap under the Hull-White one-factor model. Rather than training a surrogate to approximate the swap mark-to-market as an intermediate step, the networks are trained directly on EPE and ENE as functions of the initial yield curve, Hull-White parameters, and monitoring time. Since EPE and ENE are already expectations over the short rate distribution, the short rate does not enter the surrogate input, and CVA and DVA are recovered at inference through a single forward pass per monitoring date with no simulation. A mark-to-market surrogate is developed first as a validation step to confirm that the Hull-White pricing pipeline is correctly specified before the exposure targets are introduced.

Standard surrogate training, which optimises on price or exposure labels alone, produces networks that price accurately in-distribution but reproduce sensitivities poorly. [63] demonstrate this directly for European bond options under the Hull-White and G2++ models, across all three surrogate paradigms evaluated, strong in-distribution pricing accuracy does not translate into reliable autodiff-based Vega estimates, with degradation most pronounced near at-the-money regions and under out-of-distribution volatility shifts. The reason is structural. Minimizing a price loss imposes no constraint on the gradient of the learned function, so the network is free to interpolate prices correctly while producing gradients that bear little resemblance to the true sensitivities of the underlying model.

In the XVA setting, this matters considerably, because curve sensitivities feed directly into hedging decisions and regulatory capital calculations, and a surrogate that prices correctly but differentiates incorrectly is of limited practical value [30]. This gap between pricing accuracy and sensitivity fidelity motivates the use of differential training as the primary methodology in this study.

Chapter 2

Background and Literature Review

The valuation of a financial derivative proceeds through a structured sequence of operations, beginning with the construction of market-consistent term structures from observable instrument prices and ending with the numerical evaluation of the contract's risk-neutral expectation under a calibrated stochastic model. The first stage, term structure construction, involves fitting yield curves, discount curves, or volatility surfaces to a cross-section of market quotes, a process that must respect no-arbitrage constraints whilst accommodating the irregularity and incompleteness of available market data [66].

Model calibration follows, in which the parameters of a chosen stochastic model are estimated by minimizing the discrepancy between model-implied and market-observed prices across a set of liquid benchmark instruments [21, 23]. Once the model is calibrated, each derivative in the portfolio must be priced individually, with the resulting values used to compute risk sensitivities, margin requirements, and valuation adjustments that feed into downstream risk management and reporting systems. The pipeline is therefore not a single calculation but a chain of dependent computations, each of which inherits the errors and computational costs of the stage preceding it.

Each stage of this pipeline carries a high computational cost, and the burden compounds as model complexity and portfolio dimensionality increase. Model calibration requires repeated evaluation of the pricing function across many candidate parameter configurations, which is tractable for models with closed-form solutions but becomes expensive under stochastic volatility or interest rate models where each evaluation requires a numerical procedure [54]. Pricing itself, for instruments without analytical solutions, typically demands either Monte Carlo simulation or a partial differential equation solver, both of which scale poorly with the dimension of the state space and the number of time steps required for accurate discretisation [59, 34].

The approximation of high-dimensional pricing functions via classical numerical methods is further constrained by the curse of dimensionality, whereby the computational cost of grid-based or quadrature-based methods grows exponentially with the number of state variables, rendering them impractical for multi-asset derivatives or multi-factor term structure models [59]. The result is that even a single accurate valuation of a complex derivative can take minutes, which is acceptable in batch processing but incompatible with the response times required in real-time risk management.

In a live trading risk management environment, this per-instrument computational burden is compounded by the scale and frequency at which valuations must be produced. A derivatives portfolio may contain thousands of instruments across multiple asset classes, and regulatory frameworks such as the Fundamental Review of the Trading Book require that risk sensitivities be computed and reported under many simultaneous market scenarios within an intraday timeframe [29]. Recalibration of the model is itself triggered by each significant market move, meaning that the full pipeline, from curve construction through to pricing and Greeks, must be re-executed many times within a single trading session [2].

For instruments whose valuation depends on future exposure profiles, such as those subject to counterparty credit risk adjustments, the problem is further complicated by the need to simulate the portfolio's value across a grid of future dates and market states, a task that requires nested Monte Carlo simulation at a cost that scales with both portfolio size and the number of monitoring dates [31]. This combination of per-instrument cost and portfolio-level scale creates a computational bottleneck that cannot be resolved by incremental improvements to existing numerical methods alone, and it is this bottleneck that has motivated the adoption of deep learning surrogates as a structural solution.

Among the operations described above, the computation of xVAs stands out as the most computationally demanding, because it requires simulating the future value of every instrument in the portfolio across a grid of monitoring dates and risk scenarios, aggregating these values into exposure profiles, and then integrating those profiles against default intensity and loss assumptions to produce CVA and DVA estimates [4, 40]. The central computational objects in this pipeline are the Expected Positive Exposure and Expected Negative Exposure profiles, which must be evaluated at each monitoring date by taking expectations of the positive and negative parts of the portfolio value over the distribution of future market states, a task that requires nested Monte

Carlo simulation when no closed-form expression is available. This study focuses specifically on this bottleneck, targeting EPE and ENE directly as surrogate outputs, and evaluating whether deep learning methods can reproduce these profiles accurately enough to replace full simulation in the CVA and DVA computation.

The use of deep learning in derivatives pricing can be traced back to the seminar paper by [58], who in 1994 proposed learning networks as a nonparametric alternative to the Black-Scholes formula for pricing and hedging equity options. By using data-driven methods like radial basis function network and multilayer perceptrons, the authors demonstrate that these networks can successfully approximate the Black-Scholes formula without requiring specific assumptions about underlying asset dynamics. The approach was validated by showing that the network-implied delta-hedging strategy achieved performance comparable to the Black-Scholes delta on out-of-sample S&P 500 index options, despite the network not know the Black-Scholes formula.

The motivation behind the seminal paper of Hutchinson [58] was the desire to avoid imposing parametric model assumptions in the Black-Scholes pricing framework. The dominant paradigm has since shifted, modern applications do not use neural networks because the reference model is distrusted, but because solving it accurately is too slow. The issue is not that the underlying mathematical formulations are incorrect. Rather, the governing stochastic differential equations and their associated pricing PDEs frequently do not admit closed-form solutions, and the numerical methods required to solve them, whether Monte Carlo simulation or finite difference schemes, often carry a computational cost that becomes prohibitive at the scale and frequency demanded by live risk management systems.

For American and Bermudan options, where no closed-form solution is available, each pricing call requires Monte Carlo simulation or a backward induction procedure, and the need to repeat this across large portfolios under many scenarios creates a bottleneck that classical numerical methods cannot resolve within real-time constraints [59, 2]. The response in the recent literature has been to treat the neural network not as a model-free alternative but as a surrogate for a trusted reference model. The network is trained offline on synthetically generated labels and deployed online as an approximation to the model's own pricing function, preserving model-consistency whilst eliminating the per-query computational cost [29, 10].

This shift in motivation has a direct consequence for how surrogates are designed and evaluated. When the network stands in not for the model itself but for the expensive

numerical procedure the model requires, whether Monte Carlo simulation, a finite difference PDE solver, or a nested simulation loop for exposure computation, the relevant criterion becomes accuracy relative to what that procedure would have produced. The model's stochastic equations and risk-neutral pricing framework remain trusted and intact; what the surrogate replaces is the act of solving them numerically at query time. This distinction matters because it means the surrogate must reproduce not only the model's prices but also its sensitivity structure, since downstream applications such as hedging and xVA computation depend on both, and a network that prices correctly but differentiates incorrectly provides only partial utility within the risk management pipeline.

Under stochastic volatility models such as Heston and the rough Bergomi model, the calibration problem compounds this cost further. Fitting these models to an observed implied volatility surface requires minimizing the discrepancy between model-implied and market prices across a grid of strikes and maturities, which in turn demands repeated evaluation of the pricing function at each iteration of the optimization routine [54]. For rough volatility models in particular, where the variance process is driven by a fractional Brownian motion with no Markovian representation, even a single pricing call requires a full Monte Carlo simulation, making the calibration loop computationally intractable under real-time constraints [10]. The result is that model complexity and calibration frequency are in direct tension: the richer the model, the more accurately it captures observed market dynamics, but also the slower it is to solve, and the more frequently it needs to be recalibrated as the market moves.

The curse of dimensionality manifests most severely in xVA computation, where the computational challenge is not a single expensive pricing call but the repeated evaluation of an entire portfolio across a grid of future market states. Simulating CVA and DVA requires an outer Monte Carlo loop that generates future market scenarios and, at each scenario and each monitoring date, a re-pricing of every instrument in the portfolio under the simulated state, a procedure known as nested simulation whose cost scales with the product of the number of outer paths, the number of monitoring dates, and the per-instrument pricing cost [4, 40].

For portfolios containing instruments whose pricing functions have no closed-form solutions, or are path dependent, such as Bermudan swaptions or callable bonds, the inner pricing step itself requires simulation or backward induction, making the full nested procedure computationally prohibitive for any portfolio of realistic size [31]. This is the bottleneck that most directly motivates the surrogate modelling approach

adopted in this study, since a surrogate that replaces the inner pricing step with a single forward pass through a trained network eliminates the nested structure entirely.

For interest rate derivatives priced under multi-factor term structure models, the dimensionality of the state space creates an additional layer of computational difficulty. Grid-based PDE methods, which discretize the state space on a mesh, become impractical beyond two or three factors because the number of grid points grows exponentially with the number of dimensions, a phenomenon that Hutzenthaler et al. [59] identify as a fundamental obstacle to classical numerical methods in high-dimensional settings. Practitioners are therefore forced to rely on Monte Carlo methods, which avoid the grid but converge slowly and require many paths to achieve acceptable accuracy for smooth payoffs, particularly when sensitivities must also be estimated [23].

The literature responding to these computational challenges has developed several distinct paradigms, each positioning the neural network differently in relation to the reference financial model. Physics-informed neural networks embed the pricing PDE residual directly into the training objective, enabling the network to learn a solution surface without requiring a labelled dataset of model prices [6]. Deep backward stochastic differential equation methods reformulate the pricing problem via the Feynman-Kac correspondence and approximate the solution process using forward simulation combined with a neural network parameterisation [76]. Reinforcement learning and deep hedging approaches train agents to optimise hedging strategies directly from simulated or historical paths, bypassing the pricing equation altogether [19].

Generative models, including generative adversarial networks and variational autoencoders, have been applied to calibration and implied volatility surface generation, learning the geometry of the pricing problem from data rather than solving it analytically [26]. This study operates within the supervised surrogate paradigm, in which a reference model provides exact labels through closed-form or numerical evaluation and the neural network is trained to approximate those labels across the relevant input space, with the trained surrogate deployed as a fast substitute for the reference computation at inference time.

Deep neural networks substantially extended the scope of surrogate modelling in quantitative finance, enabling accurate approximation of pricing functions under stochastic volatility models, rough volatility models, and high-dimensional settings where classical numerical methods are computationally prohibitive [54, 59]. Within the fixed-income and counterparty credit risk literature, feed-forward networks have been applied to CVA

computation for portfolios of mixed derivatives, demonstrating that surrogate-based approaches can reproduce Monte Carlo exposure estimates with acceptable accuracy and considerable computational savings relative to full simulation [4].

For interest rate swap portfolios specifically, Vanhala [73] develops a data-driven approach to exposure profile computation using recurrent neural network architectures trained on simulated path data, showing that sequence models can learn exposure profiles without requiring repeated Monte Carlo evaluation at each monitoring date. Similarly, Dhandapani and Jain [31] apply neural network surrogates to future exposure computation for Bermudan options, achieving significant reductions in inference cost relative to full backward induction. Across these contributions, the consistent finding is that deep surrogates generalise to fixed-income and exposure settings with strong in-distribution accuracy, but validation remains focused on price or exposure fidelity, with sensitivity accuracy receiving comparatively little attention.

A structurally distinct class of surrogate models treats the pricing problem as learning an operator from a function space to a scalar output, replacing the finite-dimensional parameter vector with a yield curve or volatility surface as the network input. The deep operator network (DeepONet) is the canonical architecture in this setting, coupling a branch network that encodes the input function with a trunk network that encodes the query location, and was shown to approximate solution operators of parametric partial differential equations accurately across a broad range of input configurations [74]. For interest rate derivatives, this functional perspective is particularly natural: the pricing problem amounts to learning an operator from the space of initial term structures to bond or swap values across a range of contract and model parameters, and Cui et al. [27] develop this idea for general derivative pricing maps under term structure models.

Lee et al. [63] apply it directly to European bond call options under the Hull-White and G2++ models, using historical US Treasury yield curves as inputs and comparing DeepONet against PINN and DeepBSDE paradigms across pricing accuracy, Vega fidelity, and out-of-distribution robustness. Supervised operator learning achieves the strongest performance across all three criteria, but the authors identify a persistent gap: accurate prices do not automatically produce reliable autodiff-based sensitivities, particularly near the at-the-money region and under distribution shift. This finding establishes that the objective mismatch between price accuracy and sensitivity fidelity is not resolved by architectural sophistication alone, and motivates a training objective that targets sensitivities explicitly.

A recurring finding across the surrogate literature is that minimising a price loss alone produces networks that price accurately in-distribution but reproduce sensitivities poorly, a gap with direct consequences for any application that depends on derivative outputs rather than prices alone. Huge and Savine [55] provide the foundational treatment of this problem through the differential machine learning framework, which trains neural networks jointly on price labels and their analytic derivatives with respect to inputs, exploiting the fact that analytic sensitivities are available at negligible additional cost whenever the reference model admits closed-form or semi-analytic expressions. The joint training objective regularises the learned function, concentrating it on the correct gradient structure rather than allowing the network to interpolate prices through arbitrary paths in function space.

Subsequent work confirms the benefit of this approach across a range of settings: Mrad and Iassamen [65] demonstrate that incorporating differential objectives accelerates convergence and improves sensitivity estimates for standard equity options, whilst the controlled comparisons of Lee et al. [63] for bond options under Hull-White show that derivative-aware training materially reduces the objective mismatch observed in purely price-supervised surrogates. Together, these contributions establish differential training as the appropriate methodology when sensitivity fidelity is a design requirement, and directly motivate its adoption in this study.

Despite this body of work, a specific configuration remains largely unexamined: the direct use of differential deep learning surrogates to approximate expected exposure profiles for interest rate swaps, and the downstream propagation of surrogate-level accuracy into CVA and DVA estimates. Existing fixed-income surrogate studies either target mark-to-market values or, in the case of exposure-oriented work, use architectures trained on price or path labels without exploiting analytic sensitivities as training signals [73, 31].

Work on CVA computation via neural networks has demonstrated that surrogate approaches can reproduce Monte Carlo CVA for portfolios of derivatives [4], but the connection between the sensitivity fidelity of the surrogate and the accuracy of the resulting xVA estimates has not been examined systematically. This study addresses both gaps directly. Differential deep learning surrogates are trained with Expected Positive Exposure and Expected Negative Exposure as direct targets for a ten-year interest rate swap under the Hull-White one-factor model, and a downstream CVA propagation analysis translates surrogate-level accuracy metrics into xVA-level errors

interpretable within a risk-management framework.

Chapter 3

Thesis Outline

Part II

Theoretical Foundations

Chapter 4

Valuation of Financial Derivatives

This chapter establishes the mathematical framework underpinning the valuation of financial derivatives. Financial derivatives pricing heavily relies on stochastic calculus to model the random evolution of asset prices and derive prices free of arbitrage [50]. We consider a continuous-time financial market model over a finite time horizon $T < \infty$, and the uncertainty in the market is modeled by a probability space.

Theorem 1 (Probability Space):

A **Probability space** is the triple $(\Omega, \mathcal{F}, \mathbb{Q})$, it provides the mathematical structure for defining random processes and modeling the flow of information.

- Ω is the set of all market states
- $\mathcal{F}$ is a sigma-field describing the flow of information over time, represented by a filtration $\mathcal{F}_t$. It contains historical market prices up to time t
- $\mathbb{Q}$ is the probability measure (often referred to as the risk-neutral measure or real measure $\mathbb{P}$)

The filtration $(\mathcal{F}_t)_{t \in [0, T]}$ formalises this information structure by specifying precisely what is observable at each instant, ruling out any anticipation of future market realisations. Each financial underlying is then modelled as a stochastic process adapted to this filtration, so that its value at time t depends only on information available up to and including t .

Definition 1 (Stochastic Process):

A **stochastic process** is a family of random variables $\{X(t)\}_{t \in [0, T]}$ defined on the probability space $(\Omega, \mathcal{F}, \mathbb{Q})$. For each fixed $t \in [0, T]$, the map $X(t, \cdot) : \Omega \rightarrow \mathbb{R}$ is a random variable measurable with respect to $\mathcal{F}_t$. For each fixed $\omega \in \Omega$, the map $t \mapsto X(t, \omega)$ is called a **sample path** or **realisation** of the process. The process is said to be **adapted** to the filtration $(\mathcal{F}_t)_{t \in [0, T]}$ if $X(t)$ is $\mathcal{F}_t$ -measurable for every t , meaning the value of the process at time t depends only on information available up to and including t .

In a financial context, the sample paths $t \mapsto X(t, \omega)$ represent possible trajectories of an asset price, interest rate, or exchange rate over the horizon $[0, T]$, with the measure $\mathbb{Q}$ governing the likelihood of each trajectory. The adaptedness condition is economically natural it ensures the model does not anticipate future information, so the value of any process at time t is determined solely by the history of the market up to that point [13]. Not every adapted process possesses the structure required for no-arbitrage pricing, of particular importance is the martingale, which captures the idea that, under $\mathbb{Q}$, the best prediction of a process's future value is simply its current value.

Definition 2 (Martingale):

A stochastic process X_t is a **Martingale** with respect to filtration $\mathcal{F}_t$ under $\mathbb{Q}$ if the best prediction of its future value, conditioned on current information, is simply its current value

$$\mathbb{E}[X_t] < \infty \quad (4.1)$$

and,

$$\mathbb{E}[X_t | \mathcal{F}_s] = X_s \quad \text{with } s < t \quad (4.2)$$

The money market account, sometimes referred to as the bank account. Money market account represents a (locally) riskless investment, where profits is accrued continuously at the risk-free rate prevailing in the market at every instant [14]. Sometimes, the money market or bank account can sometimes be referred to as a default-free bond or a zero-coupon bond. It serves as the natural numeraire in risk-neutral pricing and must be distinguished from the zero-coupon bond, which is a tradable instrument promising a unit payment at a fixed maturity date, the money market account, by contrast, has no fixed maturity and rolls forward continuously at the prevailing short rate. We formalise its dynamics below.

Definition 3 (Bank Account):

A **Bank account (Money-market account)**, we define B_t as the value of the risk-free money market account at time t . We assume the account evolves deterministically according to the risk-free short rate r_t :

$$dB_t = r_t B_t dt, \quad B_0 = 1 \quad (4.3)$$

Where r_t is a positive function of time. As a result,

$$B_t = B_0 \exp\left(\int_0^t r_s ds\right) \quad (4.4)$$

Discounted stock prices are said to be martingales under the risk-neutral probability measure. Martingale is the probabilistic definition of a fair game, where the expected value of a portfolio's value tomorrow is its value today. This implies that a martingale has no systematic drift, the best prediction of its future value is its current value.

In the above r_t , represents the risk-free instantaneous short rate, defining the continuous growth rate of the bank account. It will later be referred to as the instantaneous spot rate or short rate. Subsequently we introduce the discount factor which should be differentiated from the zero-coupon bond, which is a tradable financial instrument in the form of Treasury bills,

Definition 4 (Discount Factor):

A **Discount factor** $D(t, T)$ between two time instants t and T , is the amount at time t that is equivalent to one unit of currency payable at time T and is given as by

$$D(t, T) = \exp\left(-\int_t^T r_s ds\right) \quad (4.5)$$

The discount factor $D(t, T)$ treats the short rate r_t as a known, deterministic function of time, providing a purely mathematical expression of the time value of money [14]. In practice, however, r_t is not directly observable and is itself a random quantity, which motivates the introduction of the zero-coupon bond as the market-traded counterpart of the discount factor [70]. The zero-coupon bond $P(t, T)$ therefore generalises $D(t, T)$ to the stochastic setting: when r_t is deterministic, the two coincide, but under a stochastic short rate, $P(t, T)$ is obtained as the conditional expectation of the discount factor under the risk-neutral measure $\mathbb{Q}$.

Definition 5 (Zero-Coupon Bond):

A T-maturity **Zero-Coupon bond** (pure discount factor) is a contract that guarantees its holder the payment of one unit of currency at time T, with no intermediate payments. The contract at time $t < T$ has a value $P(t, T)$ and at maturity has a value $P(T, T) = 1$.

The nature of the function r_t in the definition of the bank account is important it can be classified as both stochastic and deterministic. The implications of a stochastic r_t will be elaborated on when discussing the relationship between a zero-coupon bond and a discount factor.

A unique characteristic of zero-coupon bonds is that they have only one payment (sometimes called a bullet payment) associated with them. There is a relationship between the discount factor $D(t, T)$ and the price of the zero-coupon bond $P(t, T)$. The difference lies in the consideration of the risk-free instantaneous spot rate r_t . When deterministic $P(t, T) = D(t, T)$, differences are introduced when r_t is stochastic, and the relationship between zero-coupon bonds and the discount factor is expressed as follows

$$P(t, T) = \mathbb{E}^Q \left[e^{-\int_t^T r_s ds} \mid \mathcal{F}(t) \right] \quad (4.6)$$

A key challenge in practice is that zero-coupon bond prices are not directly observable in the market; they are theoretical constructs whose values must be extracted from liquid market instruments such as deposit rates, futures prices, and swap rates through a process known as curve bootstrapping [14]. To price financial derivatives, we define a trading strategy and the notion of absence of arbitrage.

Definition 6 (Trading Strategy):

A trading strategy is a predictable vector process $\phi_t = (\alpha_t, \beta_t)$, representing the number of units held in the risky asset S_t and the risk-free asset B_t respectively. The value of the portfolio V_t at time t is defined as

$$V_t(\phi) = \alpha_t S_t + \beta_t B_t \quad (4.7)$$

A trading strategy specifies, at each instant t , the composition of a portfolio in terms of the risky asset S_t and the risk-free account B_t . The predictability requirement on $\phi_t = (\alpha_t, \beta_t)$ ensures that positions are chosen based only on information available prior to time t , preventing the investor from anticipating future price movements [70]. In

this sense, the definition describes portfolio composition but places no restriction on how changes in that composition are financed. The self-financing condition imposes precisely this restriction. It requires that any change in portfolio value arise solely from changes in asset prices, with no external injection or withdrawal of capital [14]. This constraint is what makes replication economically meaningful: without it, any payoff could be trivially attained by adding funds to the portfolio, and the notion of a fair derivative price would lose content. Together, the trading strategy and the self-financing condition define the class of admissible replicating portfolios against which no-arbitrage pricing is constructed. If a self-financing strategy exists whose terminal value matches the payoff H_T of a derivative, that strategy determines the derivative's price uniquely under the no-arbitrage principle, provided the market is complete.

Definition 7 (Self-Financing Strategy):

A strategy $\phi_t = (\alpha_t, \beta_t)$ is **self-financing** if changes in the portfolio value arise solely from changes in asset prices, with no external injection or withdrawal of funds:

$$dV_t(\phi) = \alpha_t dS_t + \beta_t dB_t \quad (4.8)$$

In a complete market, the self-financing condition guarantees that any contingent claim H_T can be replicated by a self-financing strategy ϕ satisfying $V_T(\phi) = H_T$ almost surely [70]. The no-arbitrage price of the derivative is then simply the initial cost of the replicating portfolio, $V_0(\phi)$, since any deviation from this price would permit a riskless profit. This raises the question of what constitutes such an exploitable discrepancy, which is formalised by the notion of an arbitrage opportunity.

The no-arbitrage principle is the foundational economic axiom of modern derivative pricing [14]. Informally, an arbitrage opportunity is a self-financing strategy that generates a positive profit with positive probability, starting from zero initial investment and bearing no risk of loss. Rational markets cannot sustain such opportunities: if one existed, market participants would immediately exploit it, driving prices back to equilibrium and eliminating the discrepancy. The following definition makes these conditions mathematically precise.

Definition 8 (Arbitrage Opportunity):

An **arbitrage opportunity** is a self-financing trading strategy ϕ such that

- $V_0(\phi) = 0$,

- $V_T(\phi) \geq 0$ a.s.,
- $\mathbb{P}(V_T(\phi) > 0) > 0$.

The absence of such strategies is the fundamental economic principle underlying rational and arbitrage-free pricing.

The absence of arbitrage has a profound mathematical consequence: it is equivalent to the existence of a probability measure under which all discounted asset prices are martingales [47]. This equivalence bridges the economic condition of no-arbitrage with the mathematical machinery of martingale theory, and is the content of the Fundamental Theorem of Asset Pricing (FTAP).

The FTAP, established by Harrison and Kreps [47] and extended to continuous-time markets by Harrison and Pliska [48], is the central result of modern mathematical finance. It asserts the existence of a probability measure $\mathbb{Q}$, equivalent to the physical measure $\mathbb{P}$, under which all discounted asset prices are martingales. This measure, known as the risk-neutral or equivalent martingale measure (EMM), plays the role of the pricing measure: under it, every asset earns the risk-free rate on average, regardless of its individual risk profile [70]. The FTAP is therefore both an existence result and a practical pricing tool.

Theorem 2 (First Fundamental Theorem of Asset Pricing):

A financial market is free of arbitrage opportunities if and only if there exists a probability measure $\mathbb{Q}$, equivalent to $\mathbb{P}$, such that the discounted asset price processes are martingales under $\mathbb{Q}$. Formally,

$$\mathbb{E}^{\mathbb{Q}} \left[\frac{S_T}{B_T} \middle| \mathcal{F}_t \right] = \frac{S_t}{B_t} \quad (4.9)$$

The First FTAP guarantees the existence of at least one such measure $\mathbb{Q}$ but does not address uniqueness. In incomplete markets, multiple equivalent martingale measures may exist, each corresponding to a different internally consistent pricing rule for non-replicable payoffs [70]. The question of uniqueness is resolved by the Second Fundamental Theorem, which links it directly to market completeness.

Market completeness determines whether every contingent claim can be perfectly replicated by a self-financing strategy. In a complete market, the replicating portfolio is

unique and the derivative price is uniquely determined by the no-arbitrage condition [14]. In an incomplete market, replication may not be possible for all payoffs, and additional criteria beyond no-arbitrage are required to select a unique price. The Second Fundamental Theorem characterises completeness precisely in terms of the uniqueness of the equivalent martingale measure.

Theorem 3 (Second Fundamental Theorem of Asset Pricing):

A financial market is complete if and only if the equivalent martingale measure $\mathbb{Q}$ is unique.

Under the combined assumptions of no-arbitrage and market completeness, the price of any derivative with payoff H_T at time T is uniquely given by the expectation of the discounted payoff under $\mathbb{Q}$:

$$V_t = \mathbb{E}^{\mathbb{Q}} \left[e^{-\int_t^T r_u du} H_T \middle| \mathcal{F}_t \right] \quad (4.10)$$

Computing this expectation in practice requires transforming the dynamics of the underlying from $\mathbb{P}$ to $\mathbb{Q}$, a change of measure that is effected by the Radon-Nikodym derivative.

The change of measure from $\mathbb{P}$ to $\mathbb{Q}$ is made rigorous by the Radon-Nikodym theorem. When two probability measures are equivalent, the theorem guarantees the existence of a strictly positive random variable, the Radon-Nikodym derivative or pricing kernel, that relates expectations under one measure to those under the other [70]. In the asset pricing context, this derivative represents the stochastic discount factor adjusting for the difference in risk preferences between the physical and risk-neutral worlds. Girsanov's theorem then applies this machinery to Brownian motion, allowing the drift of a diffusion process to be altered when passing from $\mathbb{P}$ to $\mathbb{Q}$, without changing its volatility.

Theorem 4 (Radon-Nikodym Derivative):

If $\mathbb{Q}$ is equivalent to $\mathbb{P}$, there exists a strictly positive random variable ξ_T , called the **Radon-Nikodym derivative** or pricing kernel, such that for any event $A \in \mathcal{F}_T$:

$$\frac{d\mathbb{Q}}{d\mathbb{P}} = \xi_T \quad (4.11)$$

where $\xi_t = \mathbb{E}^{\mathbb{P}}[\xi_T | \mathcal{F}_t]$ is a martingale under $\mathbb{P}$.

The Radon-Nikodym derivative provides an explicit mechanism for converting expectations under $\mathbb{Q}$ into expectations under $\mathbb{P}$ and vice versa [14]. In the context of interest

rate derivatives, however, discounting with the money market account B_t introduces additional complexity when the short rate r_t is itself stochastic, since both the payoff and the discount factor are then random quantities. A more tractable approach is to change the numeraire to the zero-coupon bond $P(t, T)$, which eliminates the stochastic discount factor from the pricing formula entirely.

The choice of numeraire determines the reference asset against which all other prices are expressed, and a judicious choice can substantially simplify computation. Under the risk-neutral measure $\mathbb{Q}$, the natural numeraire is the money market account B_t . For interest rate derivatives with a fixed payment date T , however, the zero-coupon bond $P(t, T)$ is the more convenient choice, as it eliminates stochastic discounting from the pricing formula [14]. The General Change of Numeraire theorem provides the framework for switching between any two such reference assets.

Definition 9 (General Change of Numeraire):

Let N_t be a numeraire asset (a strictly positive tradable asset) with associated martingale measure $\mathbb{Q}^N$. If we select a different numeraire M_t , the price of any traded asset normalised by M_t must be a martingale under the measure $\mathbb{Q}^M$. The relationship between expectations under the two measures is given by

$$\mathbb{E}^N \left[\frac{H_T}{N_T} \middle| \mathcal{F}_t \right] = \mathbb{E}^M \left[\frac{H_T}{M_T} \middle| \mathcal{F}_t \right] \cdot \frac{M_t}{N_t} \quad (4.12)$$

where the Radon-Nikodym derivative between the two measures is the ratio of the numeraires:

$$\frac{d\mathbb{Q}^M}{d\mathbb{Q}^N} = \frac{M_T/M_0}{N_T/N_0}$$

The most important application of this theorem in interest rate modelling is the T-Forward measure $\mathbb{Q}^T$, obtained by taking the zero-coupon bond $P(t, T)$ as numeraire. Under $\mathbb{Q}^T$, the pricing formula reduces to

$$V_t = P(t, T) \mathbb{E}^{\mathbb{Q}^T} \left[H_T \middle| \mathcal{F}_t \right] \quad (4.13)$$

removing the need to model the stochastic discount factor explicitly [14]. This simplification is fundamental to the valuation of interest rate swaps under the Hull-White model and is the pricing measure adopted throughout the methodology chapter.

Chapter 5

Interest Rate Curves and Term Structure Models

"When dealing with curves, nothing ever goes straight, and in finance, the bends reveal the risks"

The financial crisis of 2007 marked a structural break in the fundamental assumptions of interest rate modelling. Before the crisis, the prevailing assumption was that credit and liquidity risks embedded in primary interbank rates like LIBOR and EURIBOR¹ were negligible [41]. The crisis shattered this paradigm, revealing that these risks were not only present but could have systemic impacts on the prices of even the most plain vanilla financial instruments. In this chapter, we introduce the fundamentals of interest rate curves, beginning with the single-curve framework, briefly outlining the multi-curve approach and the modelling of short-term rates.

Interest rates such as LIBOR, EURIBOR and JIBAR were widely regarded as practical proxies for the risk-free rate in derivative pricing and discounting frameworks. This came with several challenges during the financial credit crunch, most notably the explosion of basis spreads [41]. The basis spread refers to the divergence in interest rates between financial instruments previously considered equivalent or risk-free proxies. A simple example is the tenor basis spread between LIBOR tenors: pre-crisis, a swap rate based on semi-annual payments of the 6-month LIBOR was essentially equal to a swap rate based on semi-annual payments of the 3-month LIBOR, but post-crisis, a significant spread developed between these tenors.

¹these rates represent the rate at which major banks estimate the cost of borrowing unsecured funds from other banks in the interbank market for a specific term

Interest rate curves are a fundamental input to the valuation of financial derivatives, encoding the time value of money across all relevant maturities [56]. They determine how future cashflows are projected and discounted to present value in a manner consistent across market participants [77]. As established in Theorem 2, the value of a financial derivative is the expectation of its discounted future payoffs under the risk-neutral measure $\mathbb{Q}$, and the curve provides the discount. Historically, this valuation relied on a single risk-free rate $r(t)$ serving simultaneously as the cost of funding and the investment return on a risk-free asset, an approach known as the single-curve framework [14].

Definition 10 (Continuously-Compounded Spot Rate):

The **continuously-compounded spot interest rate** $R(t, T)$ prevailing at time t for maturity T is defined as the constant rate at which an investment made at time t accrues continuously to yield one unit of currency at time T .

$$P(t, T) = e^{-R(t, T) \cdot \tau(t, T)} \iff R(t, T) = -\frac{\ln P(t, T)}{\tau(t, T)} \quad (5.1)$$

where $P(t, T)$ is the zero coupon bond maturing at Time T as introduced in 5, $\tau(t, T)$ is the day-count fraction between t and T .

The spot rate $R(t, T)$ and the zero-coupon bond price $P(t, T)$ carry identical information: each is a transformation of the other under the chosen compounding convention. Spot rates are often called zero rates precisely because they are derived from zero-coupon bonds and represent the pure cost of money over a single horizon, free of intermediate cashflows. The relationship above holds under continuous compounding; in practice, different market segments quote rates under different conventions, so the compounding basis must always be stated alongside the rate.

Definition 11 (Day-Count Fraction):

The **day-count fraction**, denoted by $\tau(t, T)$, is a standardized method for measuring the time interval between dates t and T as a year fraction. Common conventions include Actual/365, Actual/360, 30/360, and Actual/Actual. The choice affects computed interest amounts and is contractually specified.

Whilst spot rates pertain to investments commencing immediately, financial markets also trade forward rates: rates agreed today for investment periods that begin at a future date. These are essential for pricing forward-starting instruments and for

constructing the forward curve used in derivative valuation.

Definition 12 (Forward Rate Agreement):

A **Forward Rate Agreement (FRA)** is an over-the-counter contract designed to fix the interest rate cost on an anticipated future investment or borrowing, with the settlement rate linked to a floating reference rate.

The buyer of the contract pays the fixed FRA rate, sometimes referred to as the strike, K , to the seller, whilst the seller of the FRA pays the floating rate to the buyer. Forward Rate Agreements (FRAs) do not involve an upfront premium, which implies that they create an obligation to settle at maturity. Consequently, their present value at inception, t_0 is zero, $V^{FRA}(t_0) = 0$.

For an FRA traded at time $t \leq T_1$, two counterparties agree on a fixed rate K for the accrual period $[T_1, T_2]$ on a notional amount N . At the settlement date T_1 , the LIBOR rate $L(T_1, T_2)$ is observed and, having been unknown at trade date, becomes fixed [33, 14]. The market value of the FRA at time T_1 is given by:

$$N \left[\frac{(L(T_1, T_2) - K)\tau(T_1, T_2)}{1 + L(T_1, T_2)\tau(T_1, T_2)} \right] \quad (5.2)$$

While the theoretical price of FRA at time t is defined as

$$V_t^{FRA} = NP(t, T_2) [F(t, T_1, T_2) - K] \tau(T_1, T_2) \quad (5.3)$$

In market practice, FRAs are quoted in “ $n \times m$ ” format, specifying an accrual period that *starts* n months after the spot date and *ends* m months after the spot date, with $0 < n < m$ [33]. The forward rate $F(t, T_1, T_2)$ is not a prediction of $L(T_1, T_2)$ at time t ; it is the fair break-even rate that eliminates arbitrage between investing directly and investing short-term whilst rolling over at the forward rate [67].

Definition 13 (Simple Compounded Forward Rate):

The **Simple compounded forward rate** $F(t, T_1, T_2)$ prevailing at time t for the expiry $T_1 > t$ and maturity $T_2 > T_1$ is defined by

$$F(t, T_1, T_2) = \frac{1}{\tau(T_1, T_2)} \left(\frac{P(t, T_1)}{P(t, T_2)} - 1 \right) \quad (5.4)$$

In a single curve framework, the no-arbitrage relationship between spot and forward rates follows directly

$$P(t, T_1) = P(t, T_2) \cdot (1 + F(t, T_1, T_2) \cdot \tau(T_1, T_2)) \quad (5.5)$$

The bond decomposition above is a model-free no-arbitrage identity: it holds under any term structure model that precludes arbitrage and requires no assumptions about the dynamics of the short rate $r(t)$. The forward rate $F(t, T_1, T_2)$ is thereby pinned uniquely as the ratio of observable zero-coupon bond prices, making it an unambiguous market quantity at any time $t \leq T_1$. Yet this representation is static. To price the FRA prior to T_1 , and to understand how $F(t, T_1, T_2)$ evolves as information accumulates, we must characterise its behaviour as a stochastic process across time.

The key insight is that the natural probability measure under which $F(t, T_1, T_2)$ is a martingale is not the risk-neutral measure $\mathbb{Q}$, but the T_2 -forward measure $\mathbb{Q}^{T_2}$, obtained by replacing the money market account with the T_2 -maturity zero-coupon bond $P(\cdot, T_2)$ as numeraire, as formalised in Definition 9. This change of numeraire absorbs the stochastic discount factor into the measure itself, converting the FRA valuation problem into a martingale expectation under $\mathbb{Q}^{T_2}$. The following lemma establishes this result precisely and provides the probabilistic foundation on which all subsequent interest rate pricing in this chapter rests.

Lemma 1 also has immediate structural consequences beyond the FRA. Because an interest rate swap decomposes into a portfolio of consecutive FRAs (each with its own accrual period), the value of every floating payment can be expressed as a conditional expectation under the corresponding forward measure, and the fair fixed rate of the swap emerges as a weighted average of the underlying forward rates. We formalise the interest rate swap and its valuation immediately after the proof.

Lemma 1 (Martingale Property of the Forward Rate):

Let $0 \leq t \leq T_1 < T_2$ and let $L(T_1, T_2)$ denote the (spot) forward rate set at time T_1 for the accrual period $[T_1, T_2]$, quoted on a simple-interest basis with year fraction $\tau(T_1, T_2)$. Let $P(t, T)$ denote the price at time t of a default-free zero-coupon bond maturing at T , and suppose the short rate process $r(\cdot)$ is such that

$$P(t, T) = \mathbb{E}^{\mathbb{Q}} \left[e^{-\int_t^T r(s) ds} \middle| \mathcal{F}_t \right] \quad (5.6)$$

under the risk-neutral measure $\mathbb{Q}$ as we defined in 5. Define the T_2 forward measure $\mathbb{Q}^{T_2}$ via the general change numéraire $P(\cdot, T_2)$ using 9.

The forward rate calculated at time t for the period $[T_1, T_2]$ is

$$F(t; T_1, T_2) = \frac{1}{\tau(T_1, T_2)} \left(\frac{P(t, T_1)}{P(t, T_2)} - 1 \right) \quad (5.7)$$

This implies that the forward rate $F(t, T_1, T_2)$ is always given by the expectation of the LIBOR rate $L(T_1, T_2)$ in the measure determined by $P(\cdot, T)$ where T is the expiry date of the FRA

$$F(t; T_1, T_2) = \mathbb{E}^{\mathbb{Q}^{T_2}} [L(T_1, T_2) | \mathcal{F}_t] \quad (5.8)$$

and, in particular, $F(\cdot; T_1, T_2)$ is a $\mathbb{Q}^{T_2}$ -martingale. Consequently, the fair FRA strike at time t for settlement at T_2 equals $K^* = F(t; T_1, T_2)$.

Lemma 1 establishes two related results. The first is a representation result: the forward rate $F(t; T_1, T_2)$, already defined as a ratio of zero-coupon bond prices, coincides with the conditional expectation of the realised LIBOR rate $L(T_1, T_2)$ under the T_2 -forward measure $\mathbb{Q}^{T_2}$ [14]. The second, which follows from the first, is that $F(\cdot; T_1, T_2)$ has no systematic drift under $\mathbb{Q}^{T_2}$, so its best prediction at any time t is its current value, with no adjustment for time value or risk premium. Together, these results deliver a direct pricing consequence: the fair FRA strike is simply $F(t; T_1, T_2)$, observable from quoted bond prices, independently of any assumed dynamics for the short rate $r(t)$ [67]. The proof that follows establishes this formally through a no-arbitrage argument combined with the change-of-numeraire theorem (Definition 9), from which the bond-ratio representation of the forward rate falls out as a natural consequence, setting the stage for interest rate swap valuation in Definition 14.

Proof. Consider a standard forward rate agreement (FRA) initiated at time $t \leq T_1$ with notional N , strike K , and settlement at T_2 . Under the simple-interest convention for forward rates its time- T_2 payoff is

$$\Pi_{T_2} = N \tau (L(T_1, T_2) - K) \quad (5.9)$$

Under the absence of arbitrage, the time- t value of the FRA must be zero when K equals the fair forward rate. Pricing under $\mathbb{Q}$ gives

$$0 = \mathbb{E}^{\mathbb{Q}} \left[e^{-\int_t^{T_2} r(s) ds} \Pi_{T_2} \middle| \mathcal{F}_t \right] = N \tau \mathbb{E}^{\mathbb{Q}} \left[e^{-\int_t^{T_2} r(s) ds} (L(T_1, T_2) - K) \middle| \mathcal{F}_t \right] \quad (5.10)$$

Since K is $\mathcal{F}_t$ -measurable, (5.10) implies

$$K \cdot \mathbb{E}^{\mathbb{Q}} \left[e^{-\int_t^{T_2} r(s) ds} \middle| \mathcal{F}_t \right] = \mathbb{E}^{\mathbb{Q}} \left[e^{-\int_t^{T_2} r(s) ds} L(T_1, T_2) \middle| \mathcal{F}_t \right] \quad (5.11)$$

Using $P(t, T_2) = \mathbb{E}^{\mathbb{Q}}[e^{-\int_t^{T_2} r(s) ds} | \mathcal{F}_t]$, we obtain

$$K P(t, T_2) = \mathbb{E}^{\mathbb{Q}}[e^{-\int_t^{T_2} r(s) ds} L(T_1, T_2) | \mathcal{F}_t]. \quad (5.12)$$

Define the T_2 -forward measure $\mathbb{Q}^{T_2}$ through the Radon–Nikodym density as defined in 4

$$\frac{d\mathbb{Q}^{T_2}}{d\mathbb{Q}} \Big|_{\mathcal{F}_{T_2}} \propto \frac{e^{-\int_0^{T_2} r(s) ds}}{P(0, T_2)}, \quad \text{so that} \quad \frac{1}{P(t, T_2)} \mathbb{E}^{\mathbb{Q}}[e^{-\int_t^{T_2} r(s) ds} X | \mathcal{F}_t] = \mathbb{E}^{\mathbb{Q}^{T_2}}[X | \mathcal{F}_t],$$

for any integrable X measurable at T_2 . Dividing both sides of (5.12) by $P(t, T_2)$ and applying the preceding identity with $X = L(T_1, T_2)$ yields

$$K = \mathbb{E}^{\mathbb{Q}^{T_2}}[L(T_1, T_2) | \mathcal{F}_t]$$

Hence, the fair strike is $K^* = \mathbb{E}^{\mathbb{Q}^{T_2}}[L(T_1, T_2) | \mathcal{F}_t]$. It remains to link this to the bond-price representation. Under the simple-interest convention for forward rates,

$$1 + \tau L(T_1, T_2) = \frac{P(T_1, T_1)}{P(T_1, T_2)} = \frac{1}{P(T_1, T_2)}.$$

taking conditional expectations at time t under $\mathbb{Q}^{T_2}$ and using the defining property of the T_2 -forward measure (that $B(\cdot, T_2)^{-1}B(\cdot, T_1)$ is a $\mathbb{Q}^{T_2}$ -martingale), one obtains

$$1 + \tau F(t; T_1, T_2) = \mathbb{E}^{\mathbb{Q}^{T_2}}[1 + \tau L(T_1, T_2) | \mathcal{F}_t] = \mathbb{E}^{\mathbb{Q}^{T_2}}\left[\frac{1}{P(T_1, T_2)} \Big| \mathcal{F}_t\right] = \frac{P(t, T_1)}{P(t, T_2)}.$$

Therefore,

$$F(t; T_1, T_2) = \frac{1}{\tau} \left(\frac{P(t, T_1)}{P(t, T_2)} - 1 \right) = \mathbb{E}^{\mathbb{Q}^{T_2}}[L(T_1, T_2) | \mathcal{F}_t],$$

which proves both the representation and the martingale property under $\mathbb{Q}^{T_2}$. $\square$

The proof derives the result entirely from two inputs: the no-arbitrage condition that a fairly struck FRA has zero initial value, and the change of numeraire from the money market account to the T_2 -maturity bond. No model for the dynamics of $r(t)$ is required at any step, since the forward rate $F(t; T_1, T_2)$ is pinned purely by the ratio of observable bond prices $P(t, T_1)/P(t, T_2)$, making it a model-free quantity [14]. The martingale property under $\mathbb{Q}^{T_2}$ is not an additional assumption but an automatic consequence of taking $P(\cdot, T_2)$ as numeraire. This result generalises naturally to instruments with multiple payment dates: an interest rate swap can be decomposed into a portfolio of consecutive FRAs, and the value of each floating payment follows by applying the same forward-measure argument to its respective accrual period [67]. The interest rate

swap, defined next, is the central instrument of this study, and its valuation under the Hull-White model rests directly on the martingale representation established here.

Definition 14 (Interest Rate Swap):

An **Interest Rate Swap (IRS)** is an OTC agreement between two counterparties to exchange a fixed leg (a stream of payments depending on the fixed rate) for a floating leg (a set of payments depending on the observed floating reference rate) on a notional amount constant over the contract life. The tenor of the floating rate index is the same as the payment frequency of the floating leg.

Swaps are traded over-the-counter, and the most commonly traded type is the plain vanilla swap. The difference between the value of the fixed leg and the floating leg then gives the value of IRS. A swap is said to be at par when the value of the floating leg is equal to the value of the fixed leg. The market quotes for IRS are referred to as par coupon rates. These rates represent the fixed-leg coupon that equates the present value of the swap to zero $V^{IRS}(t_0) = 0$ [5].

To elaborate on the market dynamics involved in pricing an interest rate swap, we first note that the swap payer (the party taking the long position) pays the fixed rate and receives the floating rate. For an interest rate swap, we introduce the following notation:

- N is the swap notional.
- K is the fixed coupon (swap rate).
- The fixed leg pays on dates $\{T_0^K, T_1^K, \dots, T_j^K\}$.
- The floating leg pays on dates $\{T_0^L, T_1^L, \dots, T_i^L\}$, with forward rates $L(T_{u-1}^L, T_u^L)$ observed at the reset times.

The market value of the fixed leg payment at time T_i^K is given by:

$$NK\tau(T_{i-1}^K, T_i^K), \quad (5.13)$$

where $\tau(T_{i-1}^K, T_i^K)$ denotes the accrual factor for the fixed period.

Similarly, at each floating leg payment date T_j^L , the cash flow is:

$$NL(T_{j-1}^L, T_j^L)\tau(T_{j-1}^L, T_j^L), \quad (5.14)$$

where $L(T_{j-1}^L, T_j^L)$ is the forward LIBOR rate applicable for the period (T_{j-1}^L, T_j^L) .

To compute the theoretical value of a swap at time t , we take the expected discounted value of all future cash flows occurring at their respective payment dates and then discount them back to the present. At the initial valuation time t_0 , this condition can be expressed as

$$\sum_{i=1}^N \delta(T_{i-1}, T_i) P(t, T_i) \cdot K = \sum_{j=1}^M \mathbb{E}^{\mathbb{Q}} \left(e^{-\int_t^{T_j} ds r(s)} \tau(T_{j-1}, T_j) L(T_{j-1}, T_j) \middle| \mathcal{F}_t \right) \quad (5.15)$$

An IRS can be viewed as a portfolio of consecutive FRAs, all settled at a common fixed swap rate. Consequently, the swap rate applicable over a given horizon represents a weighted average of the underlying forward rates corresponding to those FRAs. Using this fact,

$$K = \frac{\sum_{j=1}^N \tau_j P(t, T_j) F(t; T_{j-1}, T_j)}{A_N^{\text{fix}}(t)} = \sum_{j=1}^N w_j F(t; T_{j-1}, T_j), \quad (5.16)$$

$$A_N^{\text{fix}}(t) = \sum_{j=1}^N \tau_j P(t, T_j), \quad w_j := \frac{\tau_j P(t, T_j)}{A_N^{\text{fix}}(t)}. \quad (5.17)$$

Single Curve Framework

The single-curve framework rests on three results already established in this dissertation. The Fundamental Theorem of Asset Pricing (Theorems 2 and 3) guarantees that, under the risk-neutral measure $\mathbb{Q}$, every derivative is valued as an expectation of its discounted payoff with respect to the money market account B_t . The change-of-numeraire theorem (Definition 9) and Lemma 1 establish that forward rates are martingales under their respective forward measures, removing the stochastic discount factor from FRA and IRS valuation.

The bootstrapping algorithm below then translates these theoretical discount factors into a market-consistent curve by enforcing a no-arbitrage constraint: the curve must exactly reprice every calibration instrument, so that forward rates and discount factors remain internally consistent [14]. All of this rests on the assumption that a single short rate $r(t)$ governs both discounting and the projection of floating cash flows, an idealisation that held prior to 2008 but was invalidated when tenor basis spreads between LIBOR tenors became economically significant [11], a limitation the multi-curve framework addresses.

Definition 15 (Zero-Coupon Curve):

The **zero-coupon curve** (also referred to as the yield curve) is a graph of the function mapping maturities into rates at time t :

$$R(t, T) = \begin{cases} L(t, T) & t < T \leq t+1 \text{ years} \\ Y(t, T) & T > t+1 \text{ years} \end{cases} \quad (5.18)$$

The zero-coupon curve is constructed from market data through a process called bootstrapping. For maturities up to one year, money market rates $L(t, T)$ are used; for longer maturities, swap rates $Y(t, T)$ are employed. In the OTC market, rates are quoted as bids and offers, and when bootstrapping, practitioners often use the mid curve, which is the average of the two. This process requires stitching together different instruments that dominate specific time buckets. The following liquid instruments are used:

- **Short-End (Deposits):** The very short end of the curve (e.g., overnight to 1 year) is anchored by Money Market Deposits (e.g., ON, TN, SN deposits).
- **Middle Tenor (FRAs and Futures):** The middle section (typically up to 2 years) is constructed using Forward Rate Agreements (FRAs) or Interest Rate Futures.
- **Long-End (Swaps):** The long end (up to 10 or 30 years) is derived from Interest Rate Swaps (IRS).

In the above selection of liquid instruments, practitioners must employ priority rules when instruments overlap in maturities to ensure the curve is built from the most liquid data points [11]. A critical aspect of bootstrapping is the selection of an appropriate interpolation scheme. Interpolation is not only used to smooth the curve after construction, but it is also required during the bootstrapping process to solve for discount factors at dates where no direct market exists [1]. Table ?? provides an overview of the principal characteristics of the interpolation methods considered, including their structural attributes, degree of locality, forward-rate stability, and the locality of hedging adjustments, in line with the framework presented in Hagan and West [45].

Yield curve type	Forwards positive?	Forward smoothness	Method local?	Forwards stable?	Bump hedges local?
Linear on discount	no	not continuous	excellent	excellent	very good
Linear on rates	no	not continuous	excellent	excellent	very good
Raw (linear on log of discount)	yes	not continuous	excellent	excellent	very good
Linear on the log of rates	no	not continuous	excellent	excellent	very good
Piecewise linear forward	no	continuous	poor	very poor	very poor
Quadratic	no	continuous	poor	very poor	very poor
Natural cubic	no	smooth	poor	good	poor
Hermite/Bessel	no	smooth	very good	good	poor
Financial	no	smooth	poor	good	poor
Quadratic natural	no	smooth	poor	good	poor
Hermite/Bessel on rt function	no	smooth	very good	good	poor
Monotone piecewise cubic	no	continuous	very good	good	good
Quartic	no	smooth	poor	very poor	very poor
Monotone convex (unameliorated)	yes	continuous	very good	good	good
Monotone convex (ameliorated)	yes	continuous	good	good	good
Minimal	no	continuous	poor	good	very poor

Table 5.1: Interpolation Methods

Hagan and West [45] highlight that the choice of interpolation scheme is critical not just for smoothness but also for preventing arbitrage opportunities. The structure of the forward rate curve $f(t)$ is the primary visual and mathematical indicator of the quality of an interpolation method. This is because a negative forward rate implies that the discount factor function $Z(t)$ is not monotonically decreasing, which allows for arbitrage opportunities [45].

Given this segmentation, the construction of the continuous discount curve $P(t, T)$ proceeds via a recursive algorithm that solves for discount factors sequentially, moving from the shortest maturities to the longest. Let t denote the valuation (reference) date. The market provides a set of quoted instruments (deposits, FRAs, swaps) with strictly increasing terminal dates $\{d_i\}_{i=1}^n$, called *calibration dates*. Each instrument contributes one pricing equation whose unknown is the discount factor $P(t, d_i)$.

- Step 1: Initialization set $P(0, 0) = 1$
- Step 2: The curve is extended using deposits and FRAs. For the first instruments (typically up to 1 year), calculate $P(0, T_i)$ directly from the forward rate $F(0, T_{i-1}, T_i)$

$$P(0, T_i) = \frac{P(0, T_{i-1})}{1 + F(0, T_{i-1}, T_i)\tau(T_{i-1}, T_i)} \quad (5.19)$$

- Step 3: Assume the discount curve has been constructed up to time T_{k-1} . To solve for the discount factor at T_k using a par swap with fixed rate S_k , we use the par swap equation. In a single-curve framework, the present value of the floating leg equals $1 - P(0, T_k)$ (under the standard no-notional-exchange logic). Equating

floating and fixed legs gives

$$1 - P(0, T_k) = S_k \sum_{i=1}^k \tau_i P(0, T_i), \quad (5.20)$$

where α_i are the accrual fractions for the fixed-leg payments. Since $P(0, T_1), \dots, P(0, T_{k-1})$ are known from previous steps, we isolate $P(0, T_k)$ from (5.20). If the swap includes intermediate payment dates T_j with $T_{k-1} < T_j < T_k$ for which discount factors are not yet explicitly solved, we express these as functions of the known curve and the unknown target $P(0, T_k)$ via the chosen interpolation operator φ :

$$P(0, T_j) = \varphi(T_j; \{(T_i, P(0, T_i))\}_{i=0}^{k-1}, P(0, T_k)), \quad T_{k-1} < T_j < T_k. \quad (5.21)$$

Substituting (5.21) into (5.20) yields a (generally) nonlinear equation in the single unknown $P(0, T_k)$, which is then solved numerically. In the common case where all payment dates are on the existing grid (i.e., no intermediate unknowns), (5.20) reduces to a linear equation with the closed-form solution

$$P(0, T_k) = \frac{1 - S_k \sum_{i=1}^{k-1} \alpha_i P(0, T_i)}{1 + S_k \alpha_k}. \quad (5.22)$$

Repeat the recursive step for successive maturities until the longest instrument is exactly repriced, yielding a complete term structure $T \mapsto P(0, T)$. Between calibration dates, obtain a continuous curve using the selected interpolation method (e.g., linear in $\ln P(0, t)$, monotone convex).

The yield curve and the discount curve obtained from bootstrapping carry the same term-structure information but in different representations. The bootstrap algorithm produces discount factors $P(t, T)$, which are the fundamental inputs for pricing. The yield curve $R(t, T)$ is simply an alternative representation of the same term-structure information, obtained by converting discount factors into annualised zero rates under a chosen compounding convention.

Definition 16 (Yield Curve):

The **Yield Curve** at time t is the graph of the function:

$$T \mapsto R(t, T) \quad \text{for } T \geq t. \quad (5.23)$$

Practitioners pay attention to the shape of the yield curve:

- **Upward-sloping (normal) yield curve:** Long-term yields exceed short-term yields,

typically reflecting expectations of future economic growth, rising interest rates, and compensation for maturity and inflation risk.

- **Flat yield curve:** Short- and long-term yields are broadly similar, usually indicating a transition phase in the economy or uncertainty about future monetary policy, often preceding turning points in the interest rate cycle.
- **Inverted yield curve:** Short-term yields exceed long-term yields, commonly interpreted as a signal of expected economic slowdown or recession, as markets anticipate falling future interest rates.
- **Humped yield curve:** Medium-term yields exceed both short-term and long-term yields, producing a hump in the curve. This shape typically arises when markets anticipate near-term interest rate increases followed by a longer-term decline, or during periods of elevated uncertainty about the medium-term economic outlook.

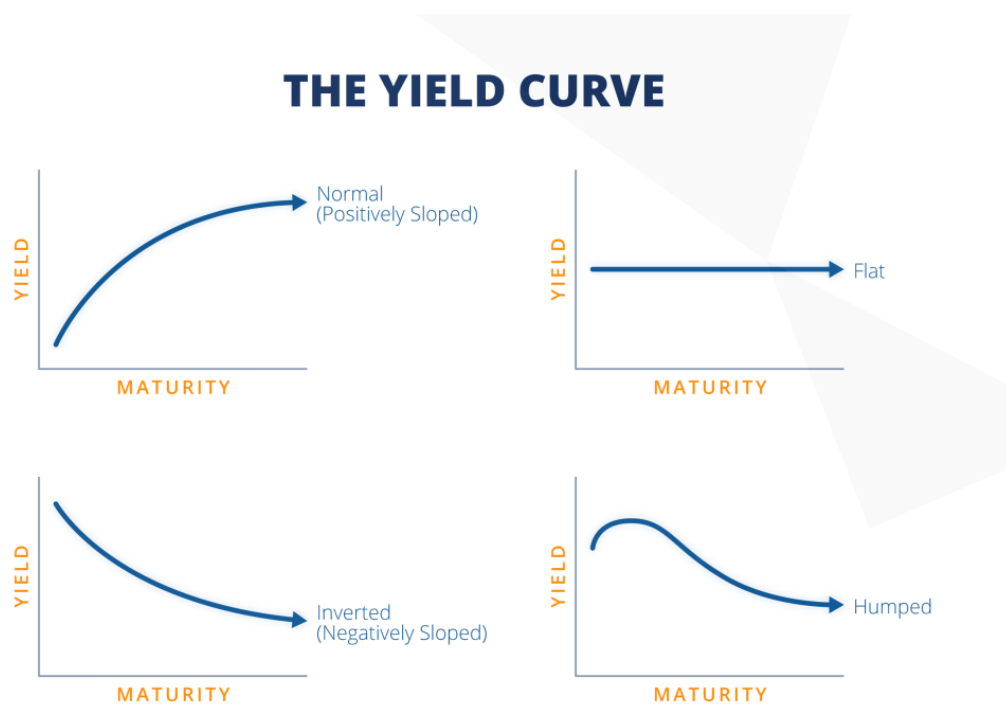


Figure 5.1: Typical yield curve shapes: upward-sloping (normal), flat, and inverted.

Parametric Curve Estimation: The Svensson Model

The bootstrapping algorithm above produces discount factors that exactly reprice all calibration instruments, but the resulting curve is piecewise in structure and sensitive

to the choice of interpolation scheme. For applications requiring a globally smooth and differentiable term structure, an alternative approach is to fit a parametric functional form directly to observed market rates. Parametric models sacrifice exact repricing in favour of parsimony and smoothness, and are widely used by central banks for monetary policy analysis and for constructing initial term structures that serve as inputs to short-rate models [7]. The most widely adopted specification is the Svensson (1994) extension of the Nelson-Siegel model, which decomposes the instantaneous forward curve into four factor loadings and two decay parameters [71].

The Svensson model specifies the instantaneous forward rate $f(0, t)$ as:

$$f(0, t) = \beta_0 + \beta_1 e^{-t/\tau_1} + \beta_2 \frac{t}{\tau_1} e^{-t/\tau_1} + \beta_3 \frac{t}{\tau_2} e^{-t/\tau_2} \quad (5.24)$$

Integrating over $[0, t]$ and dividing by t yields the corresponding continuously compounded zero rate:

$$R(0, t) = \beta_0 + \beta_1 \frac{1 - e^{-t/\tau_1}}{t/\tau_1} + \beta_2 \left(\frac{1 - e^{-t/\tau_1}}{t/\tau_1} - e^{-t/\tau_1} \right) + \beta_3 \left(\frac{1 - e^{-t/\tau_2}}{t/\tau_2} - e^{-t/\tau_2} \right) \quad (5.25)$$

The six parameters carry direct economic interpretations:

- $\beta_0 > 0$: the long-run level of interest rates, to which the curve converges as $t \rightarrow \infty$
- β_1 : the slope factor, determining the spread between short- and long-term rates; a negative value produces an upward-sloping curve
- β_2 : the first curvature factor, generating a hump or trough that peaks near maturity τ_1
- β_3 : the second curvature factor, allowing an additional hump near maturity τ_2 , extending the flexibility of the original Nelson-Siegel specification
- $\tau_1, \tau_2 > 0$: decay parameters controlling how quickly the slope and curvature contributions revert to zero with increasing maturity

The parameters are estimated by minimising the sum of squared deviations between model-implied and observed market zero rates across calibration maturities [71]:

$$\min_{\beta_0, \beta_1, \beta_2, \beta_3, \tau_1, \tau_2} \sum_i \left(R^{\text{market}}(0, t_i) - R(0, t_i) \right)^2 \quad (5.26)$$

The Svensson model is of particular relevance to the Hull-White framework developed in this study. Calibrating the time-dependent drift $\theta(t)$ to the initial term structure requires evaluating $\partial f(0, t)/\partial t$, since the exact expression for $\theta(t)$ depends on the slope of the initial forward curve at every point in time. A piecewise bootstrapped curve, being

discontinuous between calibration pillars, yields an unreliable numerical derivative at those points. The Svensson forward rate in Equation (5.24) is smooth and analytically differentiable everywhere, supplying a stable and consistent input to the Hull-White calibration described in the methodology chapter.

HJM Framework

The evolution of term structure modelling represents a structural shift in how interest rate models are formulated. Early equilibrium models, such as Vasicek (1977) and Cox-Ingersoll-Ross (1985), specified the dynamics of the instantaneous short rate $r(t)$ under a risk-neutral measure and derived the yield curve as an output [41]. This endogenous approach had a fundamental practical deficiency: because the initial yield curve was a model output rather than an input, these models could not exactly reprice the liquid benchmark instruments observed in the market, introducing systematic pricing errors that undermined their use in derivative valuation [67]. The HJM framework addressed this by reversing the modelling direction entirely.

Before stating the HJM dynamics, we formalise the object being modelled. The term structure of interest rates is not a single scalar but a continuum of rates indexed by maturity, admitting three equivalent representations.

Definition 17 (Term Structure):

The **term structure of interest rates** at time t is the collection of discount factors $\{P(t, T) : T \geq t\}$, or equivalently the family of mappings

$$T \mapsto R(t, T), \quad T \mapsto f(t, T), \quad T \mapsto P(t, T), \quad T \geq t, \quad (5.27)$$

where $R(t, T)$ is the continuously compounded zero rate, $f(t, T)$ is the instantaneous forward rate, and $P(t, T)$ is the zero-coupon bond price. Each representation encodes the same term-structure information and must be consistent with observed market prices and the no-arbitrage conditions of Theorems 2 and 3.

Given any one of the three representations, the other two are recovered without loss of information. The zero-coupon bond price $P(t, T)$ is the most fundamental, since it is the discounting kernel for future cash flows, but the instantaneous forward rate $f(t, T)$ is the most convenient state variable for modelling the dynamics of the entire curve, as it is directly observable in the limit of the simple compounded forward rate.

The Heath-Jarrow-Morton (HJM) framework [62] resolved the endogeneity problem

by taking the initial forward rate curve $f(0, \cdot)$ as a given infinite-dimensional input, calibrated directly to market data, and specifying only the stochastic dynamics of its evolution through time. This ensures an automatic, exact fit to the observed term structure at $t = 0$. Within this framework, the Hull-White (1990) model emerges not merely as an extension of Vasicek but as a specific, Markovian restriction of the general HJM dynamics that retains analytic tractability while preventing arbitrage [41].

Under the smoothness assumption on $P(t, T)$ in Definition 5, the instantaneous forward rate is well-defined for all $T \geq t$ and is related to the simple compounded forward rate $F(t, T, T + \Delta T)$ introduced in Definition 13 by taking the limit as the accrual period shrinks to zero [14].

Definition 18 (Instantaneous Forward Rate):

The **instantaneous forward rate** $f(t, T)$ prevailing at time t for maturity $T > t$ is defined by

$$f(t, T) = \lim_{\Delta T \rightarrow 0} F(t, T, T + \Delta T) = -\frac{\partial}{\partial T} \ln P(t, T) \quad (5.28)$$

The zero-coupon bond price is recovered from the instantaneous forward rate by integration:

$$P(t, T) = \exp\left(-\int_t^T f(t, u) du\right) \quad (5.29)$$

The instantaneous short rate $r(t)$ is the limiting case at the current time:

$$r(t) = f(t, t) = \lim_{T \downarrow t} R(t, T) \quad (5.30)$$

These relationships confirm that the instantaneous forward curve $T \mapsto f(t, T)$ is a complete description of the term structure: it determines all bond prices and all spot rates simultaneously.

The HJM framework models the stochastic evolution of the entire forward curve simultaneously. Under the risk-neutral measure $\mathbb{Q}$, the dynamics of $f(t, T)$ for all maturities $T \geq t$ are governed by the SDE

$$df(t, T) = \alpha(t, T) dt + \sigma(t, T) dW(t) \quad (5.31)$$

where $W(t)$ is a standard Brownian motion under $\mathbb{Q}$ and $\sigma(t, T)$ is the volatility structure. The central contribution of Heath, Jarrow, and Morton is that, under the risk-neutral measure, the drift $\alpha(t, T)$ is not a free modelling choice but is uniquely determined by the volatility structure through the no-arbitrage drift condition. For a one-factor

model,

$$\alpha(t, T) = \sigma(t, T) \int_t^T \sigma(t, u) du \quad (5.32)$$

This result means the modeller specifies only $\sigma(t, T)$ and the initial curve $f(0, T)$; the no-arbitrage requirement then determines the entire future evolution of the term structure [62]. However, for a general choice of $\sigma(t, T)$, the HJM dynamics are non-Markovian: the state of the forward curve at time t depends on the full path of W , not merely on its current value. In a lattice implementation, this produces non-recombining trees whose node count grows exponentially with the number of time steps, rendering the general framework computationally intractable for large derivative portfolios [41, 56]. The Hull-White model resolves this by imposing an economically motivated constraint on the volatility structure [56]. If $\sigma(t, T)$ is chosen to be exponentially decaying in the time-to-maturity $(T - t)$,

$$\sigma(t, T) = \sigma e^{-a(T-t)} \quad (5.33)$$

then the HJM dynamics collapse to a Markovian system driven by the single state variable $r(t)$ [62]. This volatility structure has a direct economic interpretation: shocks to the short end of the forward curve dampen at rate a as maturity increases, encoding mean reversion directly into the curve dynamics [43]. Under this specification, the short rate follows the SDE

$$dr(t) = [\theta(t) - ar(t)] dt + \sigma dW(t) \quad (5.34)$$

where $a > 0$ is the mean reversion speed, $\sigma > 0$ is the instantaneous volatility, and $\theta(t)$ is a deterministic function calibrated to fit the initial term structure exactly. This calibration requires the initial instantaneous forward curve and its time derivative, which the Svensson parametrisation introduced above supplies analytically [14]:

$$\theta(t) = \frac{\partial f^M(0, t)}{\partial t} + af^M(0, t) + \frac{\sigma^2}{2a}(1 - e^{-2at}) \quad (5.35)$$

where $f^M(0, t)$ denotes the market-implied initial instantaneous forward rate. This formulation reduces the entire state of the term structure to the single variable $r(t)$, permitting efficient simulation via recombining trinomial trees and Monte Carlo methods.

Since the Hull-White SDE is linear in $r(t)$, it admits an exact closed-form solution requiring no approximation, a property that sets it apart from nonlinear short-rate models such as Cox-Ingersoll-Ross, where explicit solutions do not exist in general [14]. The standard technique is to multiply both sides of the SDE by the integrating factor

e^{at} , which converts the left-hand side into a perfect differential via Itô's product rule:

$$d(e^{at}r(t)) = e^{at}\theta(t)dt + \sigma e^{at}dW(t) \quad (5.36)$$

Integrating both sides from s to t and dividing through by e^{at} yields the explicit solution:

$$r(t) = e^{-a(t-s)}r(s) + \int_s^t e^{-a(t-u)}\theta(u)du + \sigma \int_s^t e^{-a(t-u)}dW(u) \quad (5.37)$$

The solution decomposes the short rate into three additive components, each with a clear economic interpretation. The first term, $e^{-a(t-s)}r(s)$, is the mean-reverting decay of the current short rate: the factor $e^{-a(t-s)}$ pulls the current value exponentially towards zero at rate a , so that the influence of today's rate diminishes as the horizon lengthens. The second term is the accumulated contribution of the deterministic drift $\theta(u)$, weighted by the same exponential decay, representing the path along which the expected short rate is continuously attracted towards the market-implied forward curve. The third term is the stochastic component: a weighted sum of Brownian increments $dW(u)$ accumulated from s to t , with older shocks discounted more heavily than recent ones. In plain terms, the short rate at any future time is the net result of where it starts, where the market implies it is heading, and the random disturbances that arrive along the way [56].

Because the stochastic integral in Equation (5.37) is a linear functional of Brownian motion, it is Gaussian by construction. This means that, conditional on the filtration $\mathcal{F}_s$, the short rate $r(t)$ is normally distributed [14]:

$$r(t) | \mathcal{F}_s \sim \mathcal{N}(\mu(s, t), V(s, t)) \quad (5.38)$$

with conditional mean and variance:

$$\mu(s, t) = e^{-a(t-s)}r(s) + \int_s^t e^{-a(t-u)}\theta(u)du \quad (5.39)$$

$$V(s, t) = \frac{\sigma^2}{2a}(1 - e^{-2a(t-s)}) \quad (5.40)$$

The mean $\mu(s, t)$ reflects the two deterministic forces already identified in the explicit solution: the decayed current rate and the forward-curve-implied drift. The variance $V(s, t)$ increases monotonically from zero at $t = s$ and saturates at $\sigma^2/(2a)$ as $t \rightarrow \infty$. This saturation is a direct consequence of mean reversion: unlike a driftless Brownian motion, where uncertainty accumulates without bound, mean reversion prevents the short rate from wandering arbitrarily far from the forward curve, bounding the long-

run uncertainty by the ratio of volatility to mean reversion speed [62].

The Gaussian structure of the solution has three direct consequences for this study. First, since bond prices take the form $P(t, T) = A(t, T) e^{-B(t, T)r(t)}$ and $r(t)$ is normally distributed, bond prices are lognormally distributed. This is precisely the property that enables the closed-form ATSM expressions derived below and underpins the analytical swaption and cap pricing formulae used for calibrating a and σ to market data [14].

Second, the normal distribution assigns positive probability to negative values of $r(t)$ for all $t > s$. Negative short rates are inconsistent with the classical interpretation of the money market account as a locally riskless investment accumulating at the prevailing short rate, and represent a theoretical limitation of the Gaussian specification [41]. In practice, the probability of negative rates under calibrated Hull-White parameters is small for typical upward-sloping or flat term structures, and the analytical tractability of the Gaussian model is considered to outweigh this limitation for the IRS portfolio considered in this study.

Third, and most directly relevant to the numerical methodology, the exact conditional distribution in Equation (5.38) means that Monte Carlo simulation of the short rate requires no time-stepping approximation. At each simulation step $[t_i, t_{i+1}]$ with $\Delta t = t_{i+1} - t_i$, the short rate is drawn exactly from its conditional distribution [14]:

$$r(t_{i+1}) = e^{-a\Delta t} r(t_i) + \mu_\Delta(t_i) + \sqrt{V(t_i, t_{i+1})} Z_i, \quad Z_i \stackrel{\text{iid}}{\sim} \mathcal{N}(0, 1) \quad (5.41)$$

where $\mu_\Delta(t_i) = \int_{t_i}^{t_{i+1}} e^{-a(t_{i+1}-u)} \theta(u) du$ is the deterministic drift contribution over the step. Unlike the Euler-Maruyama scheme, which introduces a strong-order bias proportional to Δt , this discretisation is exact: the only source of error in the simulated short rate paths is Monte Carlo sampling variance, which diminishes at the standard $1/\sqrt{N}$ rate as the number of simulated paths N increases [62]. This exact simulation scheme is used directly in the exposure and xVA calculations of the methodology chapter.

A further analytical advantage of the Hull-White model is its classification as an Affine Term Structure Model (ATSM), which yields closed-form zero-coupon bond prices. Given $r(t)$, the time- t price of a zero-coupon bond maturing at T takes the exponential affine form

$$P(t, T) = A(t, T) e^{-B(t, T)r(t)} \quad (5.42)$$

where the deterministic functions $B(t, T)$ and $A(t, T)$ are given by [14]

$$B(t, T) = \frac{1 - e^{-a(T-t)}}{a} \quad (5.43)$$

$$\ln A(t, T) = \ln \frac{P^M(0, T)}{P^M(0, t)} - B(t, T) f^M(0, t) - \frac{\sigma^2}{4a} (1 - e^{-2at}) B(t, T)^2 \quad (5.44)$$

The affine structure admits closed-form expressions for European swaptions and caps, making model calibration tractable: the parameters a and σ are typically estimated by fitting to the “diagonal” of the swaption volatility matrix or a strip of at-the-money cap volatilities [14]. The principal limitation of the one-factor specification is that all instantaneous forward rates are driven by a single Brownian motion, implying perfect instantaneous correlation across the curve. This precludes exact replication of portfolios sensitive to non-parallel yield curve movements such as twists and butterflies, a limitation acknowledged in the calibration discussion of the following chapter.

Chapter 6

Counterparty Credit Risk and xVAs

The post-crisis derivatives market fundamentally altered the way in which banks price and manage OTC derivatives. Prior to 2008, the prevailing assumption in quantitative finance was that counterparties to a derivative transaction were effectively default-free, allowing the standard risk-neutral price to serve as a complete valuation [41]. The collapse of Lehman Brothers in September 2008 made explicit what had previously been treated as negligible: every OTC derivative trade carries bilateral exposure to counterparty default, and that exposure carries an economic cost that must be reflected in the price [42]. This chapter formalises the mathematical framework underlying that cost. Section 6 defines the exposure measures central to counterparty risk quantification. Section 6 introduces the legal and contractual mechanisms that reduce but do not eliminate counterparty exposure. Section 6 derives the Credit Valuation Adjustment (CVA) and Debit Valuation Adjustment (DVA) as expectation integrals over simulated exposure profiles. These integrals are evaluated numerically in Chapter 7 using the Hull-White Monte Carlo engine introduced in Chapter 5.

COUNTERPARTY CREDIT RISK

Counterparty credit risk (CCR) is the risk that a counterparty to a financial transaction will fail to fulfil its payment obligations before the final settlement of the trade [41]. Unlike loan credit risk, where exposure is one-directional and fixed at inception, the exposure arising from a derivatives transaction is bilateral and dynamic: the direction of net exposure reverses as market rates evolve, and its magnitude can grow substantially over the life of a long-dated contract [20, 42].

The central quantity in CCR measurement is the **exposure at default** (EAD), defined as the net amount owed by the defaulting counterparty at the instant of default. Since

OTC derivatives can carry both positive and negative mark-to-market value, the EAD from the perspective of party A is

$$\text{EAD}(t) = \max(V(t), 0), \quad (6.1)$$

where $V(t)$ denotes the mark-to-market value of the netted derivative portfolio at time t [42]. When $V(t) < 0$, party A owes the counterparty money and no credit exposure exists from A 's perspective at that instant.

Since default can occur at any point during the tenor of the contract, CCR management requires a forward-looking view of the exposure profile. Two expectation-based measures are standard [18, 79]. The **expected positive exposure** (EPE) and **expected negative exposure** (ENE) at time t are defined, respectively, as

$$\text{EPE}(t) = \mathbb{E}_t[\max(V(t), 0) | \mathcal{F}_t], \quad (6.2)$$

$$\text{ENE}(t) = \mathbb{E}_t[\max(-V(t), 0) | \mathcal{F}_t]. \quad (6.3)$$

Both quantities are non-negative by construction. The EPE profile captures the average exposure of party A to counterparty default, whilst the ENE profile captures the average exposure of the counterparty to party A 's own default. The EPE feeds directly into the CVA calculation and the ENE into the DVA calculation, as formalised in Section 6.

In the event of default, the non-defaulting party does not lose the full EAD. A fraction of the outstanding claim, the **recovery rate** $R \in [0, 1]$, is recovered through the bankruptcy process. The complementary quantity

$$\text{LGD} = 1 - R \quad (6.4)$$

is the **loss given default**, the true economic loss per unit of exposure [41]. In the bilateral setting of this study, separate recovery rates R_C and R_B are assigned to the counterparty and to the bank respectively, with corresponding losses given default $\text{LGD}_C = 1 - R_C$ and $\text{LGD}_B = 1 - R_B$.

A further complication in CCR arises when the exposure is correlated with the creditworthiness of the defaulting party. **Wrong-way risk** occurs when the exposure to a counterparty increases precisely as the counterparty's credit quality deteriorates, amplifying the potential loss beyond what is implied by an unconditional EPE profile [41]. For interest rate swaps, a corporate counterparty that pays fixed and receives floating may face financial stress in high-rate environments, exactly when the swap

carries positive value to the bank. **Right-way risk** is the converse: a favourable correlation between exposure and credit quality reduces realised losses relative to the unconditional EPE [42]. This study assumes independence between the short-rate process and the default intensities λ_C and λ_B , setting aside wrong-way and right-way risk effects in order to isolate the contribution of the Hull-White interest rate model to the exposure profile.

CREDIT RISK MITIGANTS

Several contractual arrangements exist to reduce counterparty credit exposure in the OTC derivatives market.

Close-out netting. A close-out netting agreement, enshrined in the ISDA Master Agreement, provides the legal basis for offsetting positive and negative mark-to-market values across a portfolio of trades between two counterparties at the moment one party defaults [41]. Without netting, the surviving party must honour trades with negative value whilst filing as an unsecured creditor for trades with positive value; under netting, only the net portfolio value constitutes the EAD. Research by ISDA suggests that netting can reduce aggregate credit exposure by as much as 85% for diversified derivatives portfolios [15].

Collateral and the CSA. A Credit Support Annex (CSA), attached to the ISDA Master Agreement, governs the periodic posting of collateral between counterparties in support of the mark-to-market of the portfolio. The call frequency, minimum transfer amount, eligible collateral types, and counterparty-specific thresholds are all specified contractually [41]. A CSA does not eliminate credit exposure entirely: the residual exposure over the margin period of risk, the interval between the last collateral posting and the close-out of the position, generates a smaller but non-zero CVA even for collateralised trades.

For the purposes of this study, we adopt the clean exposure assumption: no CSA is in place, and the full expected positive exposure profile is exposed to counterparty default. This is consistent with the bilateral, uncollateralised interest rate swap transactions that motivate the xVA calculations in Chapter 7, and it ensures that the computed CVA and DVA reflect the unmitigated bilateral credit adjustment.

CREDIT AND DEBIT VALUATION ADJUSTMENTS

The credit-risky value of a bilateral derivative differs from its default-free, risk-neutral counterpart whenever either counterparty faces a non-zero probability of default. Burgard and Kjaer [17, 16] derived the governing PDE for this bilateral credit-risky value via a replication argument under the risk-neutral measure, accounting separately for the cost of the counterparty's default through CVA and the benefit of the bank's own default through DVA.

The price of a credit risky derivative trade satisfies the following PDE

$$\begin{cases} \frac{\partial \hat{V}}{\partial t} + \mathcal{A}_t \hat{V} - r \hat{V} = \lambda_C(1 - R_C)V^+ + \lambda_B(1 - R_B)V^- \\ \hat{V}_T = V_T \text{ is the final payoff of the derivative} \end{cases}$$

where the risk-free price satisfies the Black-Scholes PDE

$$\begin{cases} \frac{\partial V}{\partial t} + \mathcal{A}_t V - rV = 0 \\ V_T \text{ is the final payoff of the derivative} \end{cases}$$

and the parabolic differential operator is given by

$$\mathcal{A}_t = \frac{1}{2}\sigma^2 S^2 \frac{\partial^2}{\partial S^2} + rS \frac{\partial}{\partial S}.$$

In the bilateral PDE, $\hat{V}$ denotes the credit-risky derivative price, V is the default-free risk-neutral price, λ_C and λ_B are the risk-neutral default intensities (hazard rates) of the counterparty and the bank, R_C and R_B are their respective recovery rates, and $V^+ = \max(V, 0)$ and $V^- = \min(V, 0)$ denote the positive and negative parts of the default-free price. The operator $\mathcal{A}_t$ is the infinitesimal generator of the underlying state process; for the Hull-White interest rate model considered in this study it takes the form $\mathcal{A}_t = [\theta(t) - ar] \partial_r + \frac{1}{2}\sigma^2 \partial_{rr}$, whilst for an equity underlying it reduces to the Black-Scholes generator as written above. The structure of the bilateral PDE is unchanged across both settings. The right-hand side decomposes into two channels: the term $\lambda_C(1 - R_C)V^+$ is the CVA source, which reduces the credit-risky price relative to the default-free price when the portfolio has positive value to the bank; and the term $\lambda_B(1 - R_B)V^-$ is the DVA source, which increases the credit-risky price when the portfolio has negative value to the bank, since the bank's own default then constitutes a benefit to the surviving counterparty.

Applying the Feynman-Kac theorem to the bilateral PDE and taking expectations under the risk-neutral measure $\mathbb{Q}$, the bilateral credit-risky price decomposes as [17, 15]

$$\hat{V}(t) = V(t) - \text{CVA}(t) + \text{DVA}(t). \quad (6.5)$$

The **Credit Valuation Adjustment** is the expected loss to the bank from the counterparty's default, weighted by the counterparty's default intensity:

$$\text{CVA}(t) = \text{LGD}_C \int_t^T \text{EPE}(s) \lambda_C(s) ds. \quad (6.6)$$

The **Debit Valuation Adjustment** is the expected gain to the bank from its own default, weighted by the bank's default intensity:

$$\text{DVA}(t) = \text{LGD}_B \int_t^T \text{ENE}(s) \lambda_B(s) ds. \quad (6.7)$$

CVA is always non-negative and reduces the bilateral price; DVA is always non-negative and increases it. The combined **bilateral credit valuation adjustment** (BCVA) is

$$\text{BCVA}(t) = \text{CVA}(t) - \text{DVA}(t), \quad (6.8)$$

so that $\hat{V}(t) = V(t) - \text{BCVA}(t)$. When $\text{BCVA} > 0$ the net credit adjustment reduces the trade value; when $\text{BCVA} < 0$ the DVA benefit dominates. The sign of BCVA depends on the relative default intensities and recovery rates of both parties and on the direction of the net exposure profile [41, 42].

The EPE and ENE integrals in equations (6.6) and (6.7) are not available in closed form for a general stochastic term structure. They are evaluated numerically in Chapter 7 by simulating the Hull-White short rate along Monte Carlo paths and computing the expected exposures at each simulation date using the closed-form IRS valuation established in Chapter 5.

Chapter 7

Neural Networks and Deep Learning

The tractability of derivative pricing under classical models rests on analytical or semi-analytical solutions that are available only under restrictive assumptions. Under the xVA framework developed in the preceding chapter, pricing must be performed across thousands of Monte Carlo simulation paths for the purpose of computing exposure profiles, and the computational cost of re-pricing a full derivatives portfolio at each time step is severe. Neural networks offer a principled response to this challenge, a trained network can emulate the pricing function to high accuracy whilst evaluating at a fraction of the computational cost of the underlying model. This chapter develops the mathematical foundations of neural networks and deep learning that underpin the surrogate modelling methodology employed in this study.

THE ARTIFICIAL NEURON

The artificial neuron is the atomic computational unit of a neural network. Inspired by the threshold-activation behaviour of biological neurons, the artificial neuron receives a vector of inputs $\mathbf{x} = (x_1, \dots, x_m)^\top$, computes a weighted sum, and produces an output determined by whether that sum exceeds a threshold. Given weights $w_1, \dots, w_m$ and threshold θ , the neuron activates if

$$h = \sum_{i=1}^m w_i x_i > \theta.$$

Reparameterising by absorbing the threshold as a bias $b = -\theta$, the activation condition becomes $\sum_{i=1}^m w_i x_i + b > 0$. The boundary $\sum_{i=1}^m w_i x_i + b = 0$ defines a hyperplane in $\mathbb{R}^m$, and the neuron partitions its input space into two half-spaces. This constitutes the linear discriminant, and it represents the simplest possible classifier [39].

The perceptron learning rule adjusts weights in response to classification errors. Given input $\mathbf{x}$, target $t \in \{0, 1\}$, and current output $y \in \{0, 1\}$, the update is

$$w_i \leftarrow w_i + \eta(t - y) x_i, \quad b \leftarrow b + \eta(t - y),$$

where $\eta \in (0, 1)$ is the learning rate, a scalar controlling the magnitude of each correction. When the prediction is correct ($t = y$), no update occurs. When the neuron fires incorrectly, the weight vector is adjusted in the direction of $\mathbf{x}$ (or its negative), geometrically moving the separating hyperplane toward the misclassified point. The algorithm converges in finite time when the dataset is linearly separable [39]. When it is not, a single neuron is insufficient: the mapping from inputs to outputs cannot be represented as a single linear discriminant, and a richer architecture is required.

MULTI-LAYER PERCEPTRONS

The limitation of the single-layer perceptron motivates the multi-layer perceptron (MLP). The XOR function illustrates the issue directly: no single hyperplane separates its positive inputs (0, 1) and (1, 0) from its negative inputs (0, 0) and (1, 1), yet two hyperplanes suffice. The MLP achieves this by introducing hidden layers: each hidden layer learns an intermediate representation, and the output layer combines those representations to produce the network's decision or prediction [69].

An MLP with L layers is specified as follows. The input layer carries the raw features $\mathbf{x} \in \mathbb{R}^{d_0}$. Each subsequent layer $k = 1, \dots, L$ receives the activation vector $\mathbf{a}_{k-1}$ from the preceding layer, applies a weight matrix $W_k \in \mathbb{R}^{d_{k-1} \times d_k}$ and a bias vector $\mathbf{b}_k \in \mathbb{R}^{d_k}$, and passes the result through an activation function g_k :

$$\mathbf{a}_k = g_k(\mathbf{a}_{k-1} W_k + \mathbf{b}_k).$$

The output layer produces $\mathbf{y} = \mathbf{a}_L$. The collection of all weights and biases across layers is denoted $\mathbf{W}$, and training amounts to finding the values of $\mathbf{W}$ that minimise a chosen measure of prediction error on the training data.

ACTIVATION FUNCTIONS

The activation function g_k introduces the non-linearity that gives multi-layer networks their expressive power. Without it, a composition of affine transformations collapses to a single affine transformation regardless of depth, and the MLP reduces to a linear

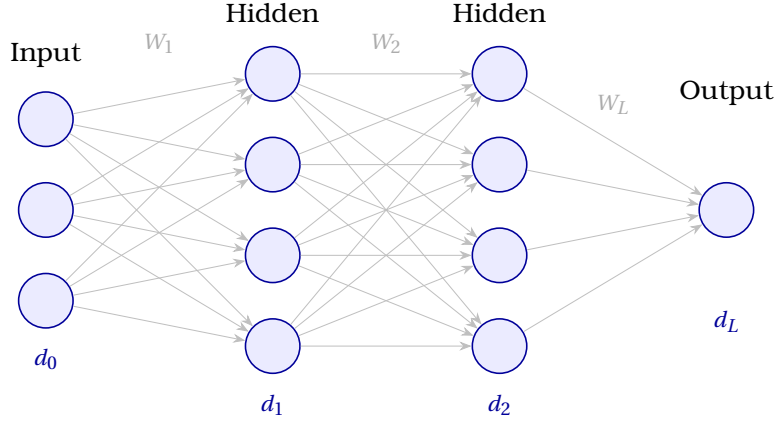


Figure 7.1: A feedforward MLP with $L = 3$ layers. Each node applies an activation function to a weighted sum of the preceding layer's outputs. Weight matrices $W_k \in \mathbb{R}^{d_{k-1} \times d_k}$ connect adjacent layers; the layer widths $d_0, \dots, d_L$ are illustrative.

model. The choice of activation function affects both the expressiveness of the network and the stability of training.

- **Sigmoid.** The logistic sigmoid $\sigma(z) = (1 + e^{-z})^{-1}$ maps $\mathbb{R}$ into $(0, 1)$ and is differentiable everywhere. Its derivative satisfies $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, a self-referential identity that simplifies gradient computation considerably. The sigmoid saturates for large $|z|$, causing gradients to vanish in deep networks and impeding learning.
- **Hyperbolic tangent.** The function $\tanh(z) = (e^z - e^{-z}) / (e^z + e^{-z})$ maps $\mathbb{R}$ into $(-1, 1)$ and is zero-centred, which tends to produce better-conditioned gradient flow than the sigmoid. Its derivative is $\tanh'(z) = 1 - \tanh^2(z)$. Like the sigmoid, $\tanh$ saturates at large magnitudes.
- **Rectified linear unit (ReLU).** The activation $\text{relu}(z) = \max(0, z)$ is piecewise linear with derivative 1 for $z > 0$ and 0 for $z < 0$ [36]. It does not saturate on the positive half-line, which mitigates the vanishing gradient problem and enables effective training of deep architectures. The sub-gradient at $z = 0$ is set to 0 by convention. ReLU is the standard choice for hidden layers in modern deep learning.
- **Linear.** The identity $\text{lin}(z) = z$ is appropriate at the output layer of a regression network, where the target is a real-valued quantity such as a derivative price and no bounding of the output is required. Using a linear output activation ensures the network can produce outputs of arbitrary magnitude and sign.

The choice of output activation is determined by the task structure. For derivative pricing, where the target is a real-valued price, a linear output activation is standard

[68]. For binary classification, a sigmoid output is appropriate, for multi-class outputs, the softmax function $\text{softmax}(z_{n_i}) = e^{z_{n_i}} / \sum_j e^{z_{n_j}}$ maps the pre-activations to a valid probability distribution over classes.

THE UNIVERSAL APPROXIMATION THEOREM

The theoretical justification for using neural networks as surrogate pricing functions is provided by the Universal Approximation Theorem. The result was established independently by Cybenko [28] for sigmoid activations and generalised to a broad class of activation functions by Hornik et al. [53], Hornik [52].

Theorem 5 (Universal Approximation):

Let $g : \mathbb{R} \rightarrow \mathbb{R}$ be a continuous, bounded, and non-constant function. For any continuous function $f : K \rightarrow \mathbb{R}$ defined on a compact set $K \subset \mathbb{R}^d$ and any $\varepsilon > 0$, there exists a single-hidden-layer neural network F_ε with activation g such that

$$\sup_{\mathbf{x} \in K} |F_\varepsilon(\mathbf{x}) - f(\mathbf{x})| < \varepsilon.$$

The theorem guarantees that a sufficiently wide single-hidden-layer network can approximate any continuous function to arbitrary precision on a compact domain. It is the composition of non-linear activations, rather than any specific sigmoid form, that confers this universal approximating capacity [52].

Three qualifications are essential. First, the theorem establishes existence but not constructibility: it provides no guidance on the network width required or on whether a particular training procedure will locate the approximating parameter configuration. Second, the result applies to compact domains; pricing functions defined over unbounded parameter ranges require additional regularity arguments. Third, approximation in theory does not imply generalisation in practice. A network may fit training inputs arbitrarily well whilst performing poorly on out-of-sample scenarios. The practical question of generalisation is addressed by training methodology and regularisation, treated in subsequent sections.

In the context of this study, Theorem 5 justifies using a neural network to emulate the Hull-White pricing function $V(r, t; \boldsymbol{\theta})$, where $\boldsymbol{\theta}$ denotes the vector of model and trade parameters. Under standard regularity conditions, this function is continuous in its arguments, and the domain of interest can be taken as a compact hypercube in the space of short-rate scenarios [65].

THE FEEDFORWARD PASS

The feedforward pass computes the network output for a given input. For node n , the pre-activation is

$$z_n = \sum_m w_{mn} y_m + b_n,$$

where the sum ranges over all nodes m in the preceding layer and y_m is the post-activation output of node m . The post-activation of node n is $y_n = g_n(z_n)$. Collecting across layers, the computation reduces to the sequential matrix propagation $\mathbf{a}_k = g_k(\mathbf{a}_{k-1} W_k + \mathbf{b}_k)$ evaluated from $k = 1$ to $k = L$.

During the forward pass, the activation values $a_n = y_n|_{\mathbf{x}, \mathbf{W}}$ are stored for subsequent use in backpropagation. Storing these intermediate values is the mechanism by which the chain rule is applied efficiently without recomputation.

LOSS FUNCTIONS

Training requires a scalar measure of the discrepancy between the network output $\mathbf{y}$ and the training target $\mathbf{t}$. This is the loss function $L(\mathbf{y}, \mathbf{t})$.

For regression tasks, the standard choice is the **mean squared error** (MSE), derived from the sum-of-squares loss:

$$L(\mathbf{y}, \mathbf{t}) = \frac{1}{2} \sum_j (y_j - t_j)^2,$$

where the sum ranges over all output nodes j [39]. MSE penalises large deviations quadratically and is appropriate when the target is a continuous quantity and errors of larger magnitude are disproportionately costly. In the surrogate pricing context, the target is the derivative price and the loss is MSE over the training portfolio.

For multi-class classification tasks with a softmax output, the cross-entropy loss is preferred:

$$L_{CE}(\mathbf{y}, \mathbf{t}) = - \sum_{i=1}^k t_i \ln(y_i),$$

where $\mathbf{t}$ is one-hot encoded and the softmax outputs form a probability distribution over classes [39]. The softmax/cross-entropy pairing admits a particularly clean gradient:

the output-layer delta simplifies to $\delta_n = a_n - t_n$, with no activation-derivative factor. This arises from the cancellation between the softmax Jacobian and the logarithmic derivative of the cross-entropy, and it makes the gradient at the output proportional to the raw prediction error.

BACKPROPAGATION

Gradient descent minimises the loss by iteratively adjusting each parameter in the direction of steepest descent. The update rule for edge weight w_{mn} is

$$w_{mn} \leftarrow w_{mn} - \eta \frac{\partial L}{\partial w_{mn}} \Big|_{\mathbf{x}, \mathbf{t}, \mathbf{W}}.$$

Computing these partial derivatives efficiently across all parameters of a multi-layer network is the purpose of the backpropagation algorithm [69]. Backpropagation is a direct application of the multivariable chain rule: gradient information is propagated backward through the network from the output layer to the input layer, accumulating the partial derivatives of each node's contribution to the loss.

The central quantity is the **delta value** at node n :

$$\delta_n = \frac{\partial L}{\partial z_n} \Big|_{\mathbf{x}, \mathbf{t}, \mathbf{W}},$$

the partial derivative of the loss with respect to the pre-activation z_n at the current weights and input. The weight gradient factorises as

$$\frac{\partial L}{\partial w_{mn}} \Big|_{\mathbf{x}, \mathbf{t}, \mathbf{W}} = \delta_n \cdot a_m,$$

so the weight update reduces to $w_{mn} \leftarrow w_{mn} - \eta \delta_n a_m$, and the bias update is $b_n \leftarrow b_n - \eta \delta_n$. The backward pass computes delta values in two stages. At each output node n :

$$\delta_n = (a_n - t_n) \cdot \frac{dy_n}{dz_n} \Big|_{\mathbf{W}}.$$

For MSE loss with linear output activation, $dy_n/dz_n = 1$, giving $\delta_n = a_n - t_n$. For MSE loss with sigmoid output, $dy_n/dz_n = a_n(1 - a_n)$, giving $\delta_n = (a_n - t_n)a_n(1 - a_n)$. At each hidden node n , the delta is propagated from the succeeding layer:

$$\delta_n = \left(\sum_{\ell} \delta_{\ell} w_{n\ell} \right) \cdot \frac{dy_n}{dz_n} \Big|_{\mathbf{W}},$$

where ℓ ranges over all nodes in the layer immediately downstream of n , and $w_{n\ell}$ is

the weight on the edge from n to ℓ . For ReLU hidden activations, $dy_n/dz_n = \mathbf{1}[z_n > 0]$; for tanh, $dy_n/dz_n = 1 - a_n^2$. The backward pass proceeds from the output layer to the first hidden layer. Once all delta values are available, all weights are updated.

RECURRENT NEURAL NETWORKS

The feedforward MLP processes each input independently, the output for a given input $\mathbf{x}$ is computed without reference to any prior inputs, and no information is carried across observations. This is appropriate when observations are exchangeable, but many problems in quantitative finance are inherently sequential. The short rate under Hull-White evolves as a continuous-time stochastic process, exposure profiles are indexed by a discrete time grid $0 = t_0 < t_1 < \dots < t_K = T$; and xVA integrals accumulate contributions across simulation time steps. An architecture that maintains state across time is required to model such sequences.

The recurrent neural network (RNN) addresses this by introducing a hidden state $\mathbf{h}_t \in \mathbb{R}^d$ that is updated at each time step t as a function of the current input $\mathbf{x}_t$ and the preceding hidden state $\mathbf{h}_{t-1}$ [39]:

$$\mathbf{h}_t = g(W_x \mathbf{x}_t + W_h \mathbf{h}_{t-1} + \mathbf{b}),$$

where $W_x \in \mathbb{R}^{n_x \times d}$ and $W_h \in \mathbb{R}^{d \times d}$ are learned weight matrices, $\mathbf{b} \in \mathbb{R}^d$ is a bias vector, and g is an activation function, typically tanh. The hidden state $\mathbf{h}_t$ serves as a compressed summary of the input history $\{\mathbf{x}_1, \dots, \mathbf{x}_t\}$, and the output at time step t is a function of $\mathbf{h}_t$. The same parameter matrices W_x , W_h , and $\mathbf{b}$ are shared across all time steps, this parameter sharing is what allows the network to process sequences of variable length and generalise across positions in the sequence.

The RNN is trained by backpropagation through time (BPTT), the network is unrolled across T time steps, producing a computational graph of depth T , and standard backpropagation is applied to this unrolled graph [39]. The gradient of the loss with respect to the hidden state at time step t involves a product of Jacobian matrices:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-k}} = \prod_{i=1}^k \frac{\partial \mathbf{h}_{t-i+1}}{\partial \mathbf{h}_{t-i}} = \prod_{i=1}^k \text{diag}(g'(W_x \mathbf{x}_{t-i+1} + W_h \mathbf{h}_{t-i} + \mathbf{b})) W_h^\top.$$

If the spectral radius of W_h is less than one, repeated multiplication causes this product to decay exponentially with k , effectively preventing the network from learning dependencies that span many time steps. If the spectral radius exceeds one, the product grows without bound and training becomes numerically unstable. These are

the vanishing gradient and exploding gradient problems respectively, and they severely limit the practical usefulness of the basic RNN for long sequences [51]. Gradient clipping addresses the exploding case, but the vanishing problem requires a structural solution: gated architectures that create direct pathways for gradient flow across time.

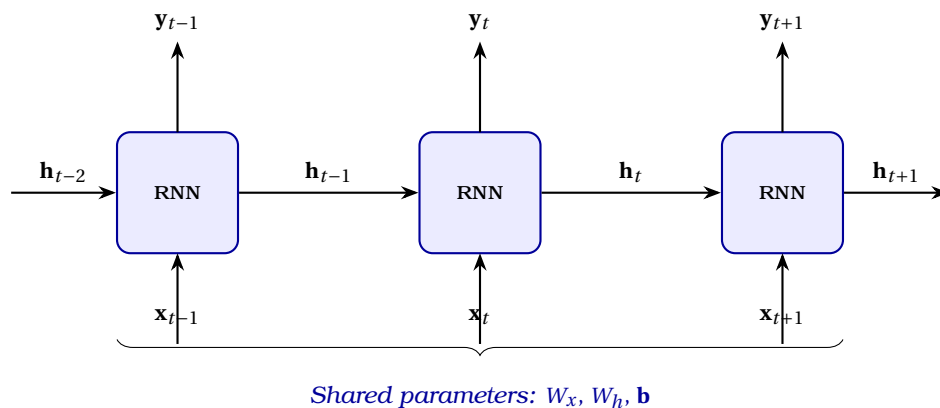


Figure 7.2: The RNN unrolled across three time steps. The recurrence $\mathbf{h}_t = g(W_x \mathbf{x}_t + W_h \mathbf{h}_{t-1} + \mathbf{b})$ applies the same parameter matrices at every step. Training by BPTT requires propagating gradients through the full unrolled graph; for a dependency spanning k steps, this involves a product of k Jacobians that decays or explodes exponentially.

LONG SHORT-TERM MEMORY NETWORKS

The vanishing gradient problem in the basic RNN was first addressed by Hochreiter and Schmidhuber [51] through the introduction of the Long Short-Term Memory (LSTM) network. The LSTM augments the hidden state with a separate **cell state** $\mathbf{c}_t \in \mathbb{R}^d$, which functions as a long-range memory channel. Information flows through the cell state with only element-wise operations across time steps, creating a direct gradient pathway that does not suffer from the exponential decay inherent in the basic RNN.

The LSTM controls information flow through four learned gate vectors. At each time step t , given input $\mathbf{x}_t \in \mathbb{R}^{n_x}$ and previous states $\mathbf{h}_{t-1}, \mathbf{c}_{t-1} \in \mathbb{R}^d$, the LSTM computes:

$$\mathbf{f}_t = \sigma(W_f \mathbf{x}_t + U_f \mathbf{h}_{t-1} + \mathbf{b}_f), \quad (7.1)$$

$$\mathbf{i}_t = \sigma(W_i \mathbf{x}_t + U_i \mathbf{h}_{t-1} + \mathbf{b}_i), \quad (7.2)$$

$$\tilde{\mathbf{c}}_t = \tanh(W_g \mathbf{x}_t + U_g \mathbf{h}_{t-1} + \mathbf{b}_g), \quad (7.3)$$

$$\mathbf{o}_t = \sigma(W_o \mathbf{x}_t + U_o \mathbf{h}_{t-1} + \mathbf{b}_o), \quad (7.4)$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \quad (7.5)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t), \quad (7.6)$$

where $W_f, W_i, W_g, W_o \in \mathbb{R}^{n_x \times d}$ and $U_f, U_i, U_g, U_o \in \mathbb{R}^{d \times d}$ are learned parameter matrices. The four gates admit the following interpretation.

The **forget gate** $\mathbf{f}_t$ determines what fraction of the previous cell state $\mathbf{c}_{t-1}$ to retain. When $\mathbf{f}_t \approx \mathbf{1}$, the cell state carries forward intact, when $\mathbf{f}_t \approx \mathbf{0}$, accumulated context is discarded.

The **input gate** $\mathbf{i}_t$ controls how much of the candidate cell state $\tilde{\mathbf{c}}_t$ is written into memory. The candidate is a tanh-activated proposal for new content, formed from the current input and the previous hidden state.

The **cell state update** (Equation (7.5)) combines the gated previous cell state and the gated candidate. Because only element-wise products and additions appear on the right-hand side, gradients can flow backward through $\mathbf{c}_t$ without repeated matrix multiplication, resolving the vanishing gradient problem for long sequences.

The **output gate** $\mathbf{o}_t$ filters the updated cell state through a tanh nonlinearity to produce the hidden state $\mathbf{h}_t$, which is the LSTM's output at time t and the quantity passed to downstream layers or the loss function.

The total parameter count for a single LSTM layer is $4(n_x d + d^2 + d)$, four times that of the basic RNN. This additional capacity improves the modelling of long-range dependencies but increases both memory requirements and training time substantially. The Gated Recurrent Unit, introduced in the following section, addresses this cost by merging the cell and hidden states into a single vector and replacing the four-gate mechanism with two gates, reducing the parameter count to $3(n_x d + d^2 + d)$ without significant loss in modelling performance [24].

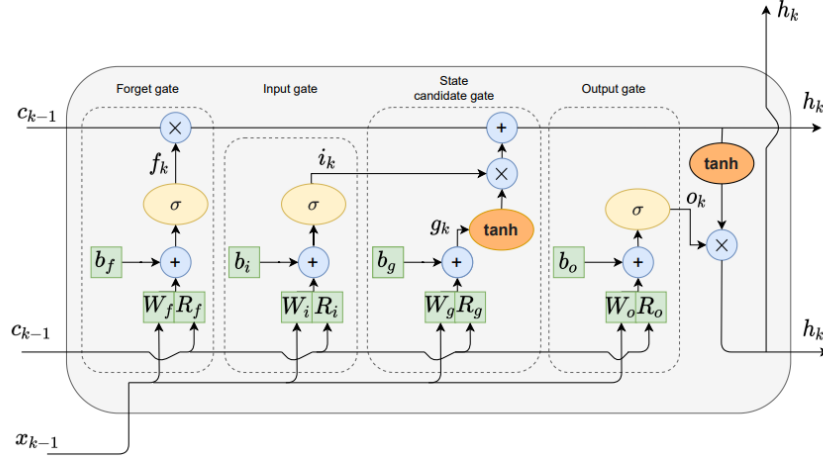


Figure 7.3: Internal dataflow of an LSTM cell at time step t . The cell state $\mathbf{c}_t$ (horizontal channel) carries long-range memory; the forget gate $\mathbf{f}_t$, input gate $\mathbf{i}_t$, and output gate $\mathbf{o}_t$ regulate what is erased, written, and read at each step. The hidden state $\mathbf{h}_t$ is derived from the cell state via the output gate.

GATED RECURRENT UNITS

The **Gated Recurrent Unit** (GRU) was introduced by Cho et al. [22] as a simplified gating architecture that mitigates the vanishing gradient problem whilst retaining fewer parameters than the Long Short-Term Memory network [51]. The GRU controls information flow through two learned gating vectors, an **update gate** $\mathbf{z}_t$ and a **reset gate** $\mathbf{r}_t$, both of which take values in $(0, 1)^d$ and operate element-wise on the hidden state.

At each time step t , given input $\mathbf{x}_t \in \mathbb{R}^{n_x}$ and previous hidden state $\mathbf{h}_{t-1} \in \mathbb{R}^d$, the GRU computes:

$$\mathbf{z}_t = \sigma(W_z \mathbf{x}_t + U_z \mathbf{h}_{t-1} + \mathbf{b}_z), \quad (7.7)$$

$$\mathbf{r}_t = \sigma(W_r \mathbf{x}_t + U_r \mathbf{h}_{t-1} + \mathbf{b}_r), \quad (7.8)$$

$$\tilde{\mathbf{h}}_t = \tanh(W_h \mathbf{x}_t + U_h (\mathbf{r}_t \odot \mathbf{h}_{t-1}) + \mathbf{b}_h), \quad (7.9)$$

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t, \quad (7.10)$$

where σ is the sigmoid function, $\odot$ denotes the Hadamard (element-wise) product, and $W_z, W_r, W_h \in \mathbb{R}^{n_x \times d}$ and $U_z, U_r, U_h \in \mathbb{R}^{d \times d}$ are learned parameter matrices. The computation in Equations (7.7)–(7.10) admits the following interpretation.

The **update gate** $\mathbf{z}_t$ governs how much of the previous hidden state is preserved in the new hidden state. When $\mathbf{z}_t \approx \mathbf{0}$, the hidden state is carried forward unchanged from

$\mathbf{h}_{t-1}$; when $\mathbf{z}_t \approx \mathbf{1}$, the hidden state is replaced entirely by the candidate $\tilde{\mathbf{h}}_t$. The network learns to open or close this gate adaptively based on the input and the current state.

The **reset gate** $\mathbf{r}_t$ controls how much of the previous hidden state is made available when computing the candidate $\tilde{\mathbf{h}}_t$. When $\mathbf{r}_t \approx \mathbf{0}$, the previous hidden state is suppressed and the candidate is computed as if from a fresh initialisation, allowing the network to discard accumulated context that is no longer relevant. When $\mathbf{r}_t \approx \mathbf{1}$, the full previous state is incorporated.

The **candidate hidden state** $\tilde{\mathbf{h}}_t$ (Equation (7.9)) is a tanh-activated combination of the current input and the gated previous state. It represents the new information that could potentially enter the hidden state at this time step.

The **final hidden state** $\mathbf{h}_t$ (Equation (7.10)) is a convex interpolation between the previous hidden state and the candidate, with interpolation weights determined by the update gate. Crucially, this interpolation creates a near-direct path from $\mathbf{h}_{t-1}$ to $\mathbf{h}_t$ whenever $\mathbf{z}_t$ is small. When unrolled across T steps, this path prevents the gradient from vanishing: the derivative $\partial \mathbf{h}_t / \partial \mathbf{h}_{t-k}$ is no longer a product of k Jacobians but instead accumulates additively, much as residual connections do in feedforward networks. This is the mechanism by which the GRU addresses the fundamental limitation of the basic RNN.

The total number of learnable parameters in a single GRU layer with input dimension n_x and hidden dimension d is $3(n_x d + d^2 + d)$, corresponding to the three sets of weight matrices and bias vectors in Equations (7.7), (7.8), and (7.9). This is two-thirds the parameter count of an equivalent LSTM layer, which maintains both a hidden state and a separate cell state [24]. Empirical comparisons indicate that the GRU matches LSTM performance on many sequence modelling tasks whilst being faster to train, a finding that motivates its use in the computationally intensive xVA context of this study [24].

For a multi-layer GRU, the hidden state $\mathbf{h}_t^{(\ell)}$ at layer ℓ takes the hidden state $\mathbf{h}_t^{(\ell-1)}$ from the layer below as its input $\mathbf{x}_t$, stacking the recurrent computation across depth. The output of the GRU at each time step t is typically a linear function of the final-layer hidden state $\mathbf{h}_t^{(L)}$, and for sequence-to-one prediction tasks (such as predicting a scalar xVA quantity from a simulated path), the hidden state at the terminal time step $\mathbf{h}_T^{(L)}$ is passed to a feedforward output layer.

OPTIMISATION

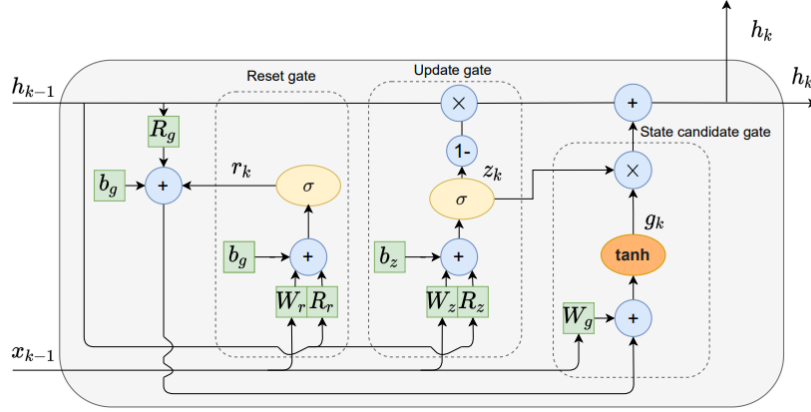


Figure 7.4: Internal dataflow of a GRU cell at time step t . The update gate z_t interpolates between the previous state $\mathbf{h}_{t-1}$ (weighted by $1 - z_t$) and the candidate $\hat{\mathbf{h}}_t$ (weighted by z_t). The reset gate $\mathbf{r}_t$ controls how much of $\mathbf{h}_{t-1}$ enters the candidate computation. The near-direct path from $\mathbf{h}_{t-1}$ to $\mathbf{h}_t$ when $z_t \approx 0$ is the mechanism that prevents gradient vanishing across long sequences.

Gradient Descent Variants

Full-batch gradient descent computes the exact gradient of the loss over all N training samples before updating weights. This is computationally expensive per step and slow to converge when N is large. **Stochastic gradient descent** (SGD) updates weights after every single training sample, approximating the gradient with a high-variance single-sample estimate. This is cheap per step but introduces substantial noise into the optimisation trajectory.

Mini-batch gradient descent balances these extremes. Weights are updated after processing a mini-batch of M samples, where $1 < M \ll N$. The averaged gradient

$$\bar{g}_{w_{mn}} = \frac{1}{M} \sum_{i=1}^M \frac{\partial L^{(i)}}{\partial w_{mn}} \Big|_{\mathbf{w}}$$

is a lower-variance estimate than the single-sample SGD gradient, whilst updates remain frequent enough to exploit gradient information efficiently. Mini-batch training also maps naturally to GPU parallelism and is the standard approach in practice [39].

Adaptive Learning Rate Methods

The Adam optimiser [60] augments mini-batch gradient descent with adaptive per-parameter learning rates. Adam maintains exponentially decaying averages of past gradients (first moment m_t) and past squared gradients (second moment v_t):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t,$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2,$$

with bias-corrected estimates $\hat{m}_t = m_t / (1 - \beta_1^t)$ and $\hat{v}_t = v_t / (1 - \beta_2^t)$. The parameter update is

$$\theta_t = \theta_{t-1} - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t,$$

with standard defaults $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$ [60]. The second moment normalises the step size per-parameter: directions with consistently large gradients are taken with smaller effective learning rates, whilst directions with small or noisy gradients are amplified. Adam is robust across a wide range of architectures and is used as the default optimiser in this study.

REGULARISATION AND GENERALISATION

Overfitting and the Bias-Variance Tradeoff

A network trained to minimise training loss can memorise the training data, including noise, and perform poorly on previously unseen inputs. This is **overfitting**. The underlying tension is between bias and variance [49]: a low-capacity model is biased toward simpler functions and may underfit (high training error regardless of data volume); a high-capacity model can represent complex functions but absorbs noise in the training data, producing high variance in predictions on new inputs.

The standard diagnostic is to monitor training loss L_T and validation loss L_V across epochs. Both decrease initially as the network learns the underlying function. When L_V begins to increase whilst L_T continues to fall, the network is fitting noise. Training is stopped at the epoch of minimum L_V : this is **early stopping**, which uses the validation set as a proxy for generalisation performance.

Weight Regularisation

Explicit regularisation modifies the loss function to penalise large weights, reducing the effective capacity of the network. The two standard forms are:

$$L_2(\mathbf{y}, \mathbf{t}) = L(\mathbf{y}, \mathbf{t}) + \frac{\lambda}{2} \sum_{\text{all } w_{mn}} w_{mn}^2,$$

$$L_1(\mathbf{y}, \mathbf{t}) = L(\mathbf{y}, \mathbf{t}) + \lambda \sum_{\text{all } w_{mn}} |w_{mn}|.$$

Under L_2 regularisation, the modified weight update includes a shrinkage term proportional to the current weight: $w_{mn} \leftarrow w_{mn} - \eta(\partial L / \partial w_{mn}) - \eta\lambda w_{mn}$. This is called **weight decay**, and its effect is to pull all weights toward zero proportionally to their magnitude, preventing any single weight from becoming excessively large. Under L_1 regularisation, the update instead subtracts a constant-sign term: $w_{mn} \leftarrow w_{mn} - \eta(\partial L / \partial w_{mn}) - \eta\lambda \text{sign}(w_{mn})$. The L_1 penalty drives small weights to exactly zero, producing sparse weight configurations. The regularisation coefficient λ is a hyperparameter tuned using the validation set [49].

Dataset Partitioning

Training a network that generalises requires discipline in how data is used. The dataset is partitioned into three disjoint subsets: the **training set**, used to update weights; the **validation set**, used to monitor overfitting and tune hyperparameters; and the **test set**, used once at the conclusion of training to report final model performance. A typical partition is 50% training, 25% validation, and 25% test, with all subsets drawn randomly from the full dataset. Using the test set during model development invalidates its role as an unbiased estimator of generalisation error.

When training data is limited, k -fold cross-validation provides more reliable generalisation estimates. The dataset is divided into k equal folds and the training-testing procedure is repeated k times, each time holding out a different fold. The performance metric is averaged over all k rounds, and every data point is used for testing exactly once [49].

DATA PREPARATION

Neural network training is sensitive to the scale of input features. When features span very different numerical ranges, the contributions of different inputs to the pre-activation at the first hidden layer are unequal. Inputs with large magnitudes dominate the activation and drive the weight updates during early training, whilst the network is slow to resolve the influence of smaller-valued inputs. Normalising all features to a common scale before training reduces this problem and improves convergence.

Two methods are standard. **Min-max normalisation** maps each feature to the interval $[0, 1]$:

$$\tilde{d} = \frac{d - d_{\min}}{d_{\max} - d_{\min}}.$$

Z-score standardisation rescales each feature to zero mean and unit variance:

$$\tilde{d} = \frac{d - \mu}{\sigma},$$

where μ and σ are the sample mean and standard deviation computed from the training set. Both transformations are invertible, so predictions in the normalised space can be mapped back to original units. Z-score standardisation is generally preferred when the input distribution is not bounded and may be approximately Gaussian [39]. In the context of this study, input features include short rates, time-to-maturity, and notionals, which span several orders of magnitude and are standardised prior to network training. Target prices are likewise normalised to improve numerical conditioning of the optimisation.

SUMMARY

This chapter has developed the theoretical machinery of neural networks and deep learning that underpins the surrogate pricing methodology of this study. Beginning with the artificial neuron and the perceptron learning rule, we have established how multi-layer architectures achieve non-linear function approximation. The Universal Approximation Theorem provides the existence guarantee that justifies applying neural networks to continuous pricing functions defined on compact domains. Backpropagation operationalises gradient descent by efficiently computing parameter gradients via the chain rule, with explicit delta formulas derived for MSE and cross-entropy losses.

The feedforward MLP, whilst theoretically expressive, treats observations as independent. For pricing problems indexed by a simulation time grid, a recurrent architecture is more natural. The Gated Recurrent Unit addresses the vanishing gradient limitation of the basic RNN through an update gate and a reset gate, which create direct gradient pathways across time steps and allow the network to selectively retain or discard accumulated context. Mini-batch optimisation with Adam, combined with L_2 regularisation and early stopping, addresses the practical challenges of training on finite datasets. Data normalisation ensures numerical stability when inputs span disparate scales.

With these foundations established, the following chapter presents the differential deep learning framework of Mrad and Iassamen [65], which augments the surrogate architecture with pathwise sensitivity information, enabling simultaneous approximation of prices and Greeks from a single trained network with substantially improved sample efficiency.

Part III

Methodology and Models

Chapter 8

Methodology

8.1 INTRODUCTION

For our implementation, we consider an investor holding a long position in a ten-year interest rate swap. This implies that the investor will pay a fixed rate on the notional principal and receive a floating rate referencing the prevailing short-term interest rate, with no exchange of principal and only the net interest cashflow settled between the two parties at each payment date. We assume that the counterparties trade in accordance with the confirmation set out in Table 8.1 below, in which Party A is designated the fixed rate payer, corresponding to the investor's long position, and Party B the floating rate payer on the other side of the transaction.

Table 8.1: Confirmation of the interest rate swap transaction, presented in the format of a standard ISDA confirmation

Field	Value
<i>A. Transaction Details</i>	
Parties	Party A and Party B
Trade Date	$t = 0$
Effective Date	$t = 0$
Termination Date	$T_n = 10$ years from the Effective Date
Calculation Agent	Party B
<i>Fixed Amounts</i>	
Fixed Rate Payer	Party A
Notional Amount	N (constant; no amortisation)
Fixed Rate	K^{par} , set at the Effective Date per the par-swap condition
Fixed Rate Day Count Fraction	30/360 (semi-annual accrual, $\tau = 0.5$)
Fixed Rate Payer Payment Dates	$T_j = j\tau$, $j = 1, \dots, 20$ (semi-annual, in arrears)
<i>Floating Amounts</i>	
Floating Rate Payer	Party B
Notional Amount	N (matches Fixed Amounts)
Floating Rate Option	Resets to the instantaneous short rate implied by the Hull-White model (single-curve valuation, no credit spread)
Spread	None
Floating Rate Day Count Fraction	30/360 (semi-annual accrual, $\tau = 0.5$)
Floating Rate Payer Payment Dates	$T_j = j\tau$, $j = 1, \dots, 20$ (semi-annual, in arrears)
Compounding	Inapplicable
<i>Other Terms</i>	
Business Day Convention	Not applicable (continuous-time model; no calendar adjustment)
Governing Model	One-factor Hull-White short-rate model, calibrated to the Svensson zero curve at $t = 0$

We develop and evaluate deep learning surrogate models as replacements for computationally expensive classical pricing routines within a Monte Carlo-based xVA framework. The objective is twofold: to achieve price approximation accuracy that is practically indistinguishable from the underlying Hull-White pricer, and to ensure that the implied

price sensitivities are recovered correctly.¹

The first is a standard Gated Recurrent Unit (GRU) network trained on price labels derived from the Hull-White analytical pricer, taking discount factor curves and model parameters as inputs. The second is a differential surrogate trained jointly on price and key-rate DV01 labels, following the differential deep learning framework of [55]. The motivation for the comparison is to establish whether joint training on sensitivities produces measurable improvements in both price and Greeks accuracy relative to price-only training.

8.2 MODEL ASSUMPTIONS

The analysis rests on a set of modelling assumptions that govern the scope and implementation of this study. These are stated here explicitly to establish a clear theoretical boundary before the technical development begins.

The initial yield curve is parameterised using the Nelson-Siegel-Svensson (1994) functional form, with parameters sourced from the Federal Reserve GSW dataset [44]. In a practice, the term structure would be constructed by bootstrapping from quoted market instruments. The Svensson curves used here serve as analytically smooth proxies, the GSW estimates are fitted daily to US Treasury yields and reproduce the principal shapes observed in real fixed income markets, including normal, humped, flat, inverted, and steep configurations. All rates are expressed on a continuously-compounded, act/365 basis.

¹We restrict our attention to the DV01, the sensitivity of the swap value to a parallel shift in the yield curve.

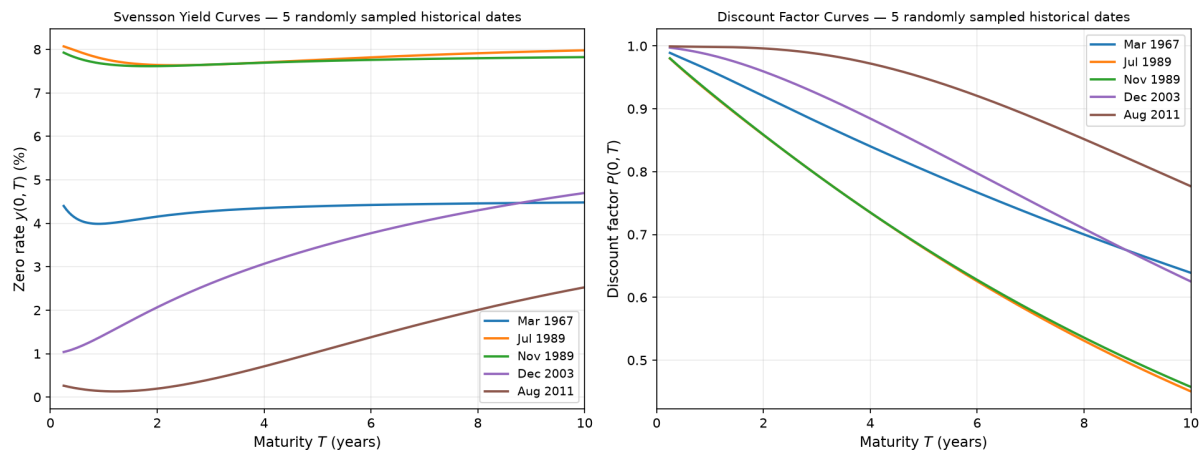


Figure 8.1: Svensson yield curves (left) and corresponding discount factor curves (right) for five randomly sampled historical dates from the Federal Reserve GSW dataset. The selection illustrates the range of term structure shapes used as proxies for market yield curve data in this study.

The short rate evolves under the risk-neutral measure $\mathbb{Q}$ according to the Hull-White one-factor dynamics, with mean reversion speed $a > 0$ and instantaneous volatility $\sigma > 0$ both held constant over the full simulation horizon. The drift function $\theta(t)$ is uniquely determined by the requirement that the model reproduce the initial term structure exactly. This study does not calibrate a and σ from market-implied swaption volatilities, both are instead treated as inputs to the surrogate and sampled across a practical range, enabling the network to approximate the pricing function over a broad region of the parameter space. The simulation is initialised at the instantaneous short rate $r_0 = \lim_{T \rightarrow 0} y(0, T) = \beta_0 + \beta_1$, which is the limiting value of the Svensson zero yield as maturity approaches zero [32].

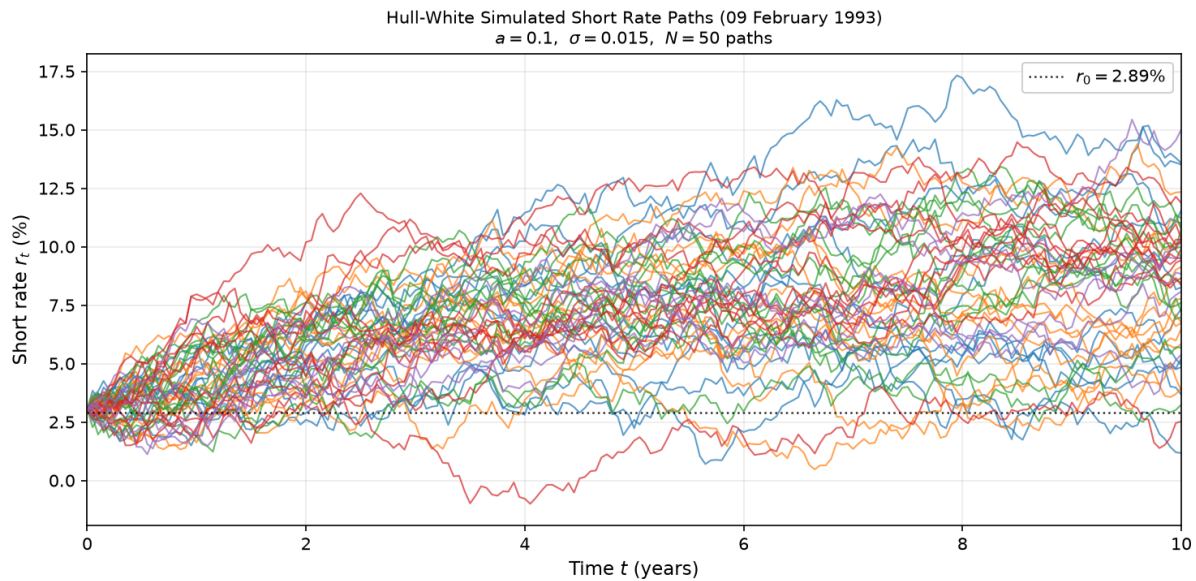


Figure 8.2: Simulated Hull-White short-rate paths over the ten-year IRS horizon, initialised at $r_0 = \beta_0 + \beta_1$ from the Svensson term structure. The upward drift of the path ensemble reflects the positive slope of the initial forward curve embedded in $\theta(t)$, consistent with an upward-sloping initial yield curve.

The underlying instrument is a ten-year payer IRS with semi-annual payment dates $T_j = j\tau$, $j = 1, \dots, 20$, and day count fraction $\tau = 0.5$. A single-curve framework is adopted, the same Svensson-calibrated curve is used for both discounting and floating rate estimation. No credit spread, margin, or collateral agreement is assumed. The swap is initiated at par, so $K = K^{\text{par}}$ and the value at inception is zero. Notional is normalised to $N = 1$, and calendar effects are disregarded throughout.

Monte Carlo simulation is conducted under $\mathbb{Q}$, with the short-rate process discretised via the Euler-Maruyama scheme. The simulation horizon is ten years, divided into 120 monthly steps so that monitoring dates $t_k = k/12$, $k = 0, 1, \dots, 120$, align exactly with the path grid. No variance reduction techniques are employed. Training labels for the surrogate models are generated analytically from the Hull-White closed-form bond pricing formulae; no simulation noise enters the training data. Sensitivity labels for Model 1b are computed via automatic differentiation, not finite differences. For the xVA calculation, only unilateral CVA is considered, counterparty default is assumed independent of exposure, the hazard rate is flat, and recovery is fixed at $R = 40\%$.

8.3 NUMERICAL PRICING OF THE IRS UNDER HULL-WHITE

The computation of Credit Valuation Adjustment requires evaluating the expected positive exposure of the IRS at each date over the full life of the contract. The Expected Positive Exposure at monitoring date t_k is:

$$\text{EPE}(t_k) = \mathbb{E}^{\mathbb{Q}}[\max(V_{\text{IRS}}(t_k, r_{t_k}), 0)] \quad (8.1)$$

Computing this across N simulated scenarios and $K = 120$ monthly monitoring dates requires the swap value $V_{\text{IRS}}(t_k, r_{t_k}^{(i)})$ at each scenario $(t_k, r_{t_k}^{(i)})$, for $i = 1, \dots, N$ and $k = 0, \dots, 120$. This is the central pricing problem: given a future state (t, r_t) with $t > 0$, determine the value of the remaining cashflows of the IRS at that state.

Under the risk-neutral measure $\mathbb{Q}$, this value is the conditional expectation of discounted future net cashflows:

$$V_{\text{IRS}}(t, r_t) = \mathbb{E}_t^{\mathbb{Q}} \left[\sum_{j: T_j > t} \exp\left(-\int_t^{T_j} r_s ds\right) \text{CF}_j \right] \quad (8.2)$$

where CF_j denotes the net cashflow at payment date T_j . Under the Hull-White model, this expectation admits two computationally distinct representations. The first evaluates it in closed form via the affine term structure, yielding an exact analytical price at any (t, r_t) . The second estimates it by Monte Carlo simulation, averaging discounted cashflows across forward paths initiated from the state (t, r_t) . Both representations evaluate the same expectation; the simulation estimate converges to the analytical price as the number of inner paths increases.

This equivalence motivates the design of the two surrogate models developed in this study. Model 1a is trained to replicate the closed-form pricer, applicable wherever an analytical solution exists. Model 1b is trained to replicate the simulation-based pricer, applicable in the more general setting where no closed form is available, as is the case for multi-factor term structure models or path-dependent payoffs. Since both pricers evaluate the same underlying expectation, their convergence provides a natural validation benchmark: agreement between the two surrogate outputs confirms that each is correctly approximating the same pricing function. The two representations are described in the subsections that follow.

Analytical Valuation: Affine Term Structure

The initial discount factors $P(0, T_j)$ at the semi-annual payment dates are computed from the Svensson zero curve:

$$P(0, T_j) = \exp(-y(0, T_j) \cdot T_j), \quad j = 1, \dots, 20 \quad (8.3)$$

At a future time $t > 0$ and short rate r_t , each bond price is computed via the affine formula:

$$P(t, T_j) = A(t, T_j) \exp(-B(t, T_j) r_t) \quad (8.4)$$

where:

$$B(t, T_j) = \frac{1 - e^{-a(T_j - t)}}{a} \quad (8.5)$$

$$\ln A(t, T_j) = \ln \frac{P(0, T_j)}{P(0, t)} - B(t, T_j) f(0, t) - \frac{\sigma^2}{4a} B(t, T_j)^2 (1 - e^{-2at}) \quad (8.6)$$

with $f(0, t)$ the initial instantaneous forward rate, approximated via central differences:

$$f(0, t) \approx -\frac{\ln P(0, t + \Delta t) - \ln P(0, t - \Delta t)}{2\Delta t} \quad (8.7)$$

The payer IRS value at (t, r_t) is then assembled as:

$$V_{\text{IRS}}(t, r_t) = P(t, T_0) - P(t, T_n) - K^{\text{par}} \tau \sum_{j=1}^n P(t, T_j) \quad (8.8)$$

with the par rate computed once at $t = 0$:

$$K^{\text{par}} = \frac{1 - P(0, T_n)}{\tau \sum_{j=1}^n P(0, T_j)} \quad (8.9)$$

All quantities are evaluated in closed form at each scenario without numerical integration or root-finding. This tractability is what makes analytical label generation feasible at the scale required for surrogate training.

Simulation-Based Valuation: Monte Carlo

The simulation-based representation of (8.2) estimates the conditional expectation directly by forward simulation. Given a state (t, r_t) , M independent short-rate paths

are simulated from time t to maturity T_n via the Euler-Maruyama scheme:

$$r_{s+\Delta s}^{(i)} = r_s^{(i)} + (\theta(s) - a r_s^{(i)})\Delta s + \sigma \sqrt{\Delta s} Z_s^{(i)}, \quad Z_s^{(i)} \sim \mathcal{N}(0, 1) \quad (8.10)$$

On each path i , the net cashflow at remaining payment date $T_j > t$ is:

$$CF_j^{(i)} = \left(\frac{1}{P(T_{j-1}, T_j | r_{T_{j-1}}^{(i)})} - 1 \right) - K^{\text{par}} \tau \quad (8.11)$$

where $P(T_{j-1}, T_j | r_{T_{j-1}}^{(i)})$ is the affine bond price evaluated at the simulated short rate $r_{T_{j-1}}^{(i)}$. Each cashflow is discounted to t using the path numeraire:

$$D^{(i)}(t, T_j) = \exp \left(- \sum_{s: t \leq s < T_j} r_s^{(i)} \Delta s \right) \quad (8.12)$$

The simulation-based swap value is then:

$$V_{\text{MC}}(t, r_t) = \frac{1}{M} \sum_{i=1}^M \sum_{j: T_j > t} CF_j^{(i)} D^{(i)}(t, T_j) \quad (8.13)$$

By the risk-neutral pricing theorem, V_{MC} is an unbiased estimator of V_{affine} , and the two converge as $M \rightarrow \infty$. This is the pricing method replicated by Model 1b.

8.4 MONTE CARLO SIMULATION AND EXPOSURE

The expected exposure profiles are computed by simulation across N independent outer paths. For each path $i = 1, \dots, N$, the short rate is evolved from $t = 0$ to T_n via (8.10) over $K = 120$ monthly monitoring dates $t_k = k/12$, $k = 0, 1, \dots, 120$. At each scenario (i, t_k) , the swap value $V_{\text{IRS}}(t_k, r_{t_k}^{(i)})$ is evaluated using the appropriate pricer. The Expected Positive Exposure and Expected Negative Exposure profiles are:

$$\text{EPE}(t_k) = \frac{1}{N} \sum_{i=1}^N \max(V_{\text{IRS}}(t_k, r_{t_k}^{(i)}), 0) \quad (8.14)$$

$$\text{ENE}(t_k) = \frac{1}{N} \sum_{i=1}^N \min(V_{\text{IRS}}(t_k, r_{t_k}^{(i)}), 0) \quad (8.15)$$

These profiles feed directly into the CVA calculation in Section 8.6. In the surrogate-based calculation, $V_{\text{IRS}}(t_k, r_{t_k}^{(i)})$ is replaced by the surrogate output $\hat{V}(t_k, r_{t_k}^{(i)})$, reducing each scenario valuation to a single network forward pass. The accuracy of this substitution, measured by the gap between the surrogate-based and reference EPE profiles,

is one of the primary evaluation criteria in the Results chapter.

8.5 SURROGATE MODELS

Three surrogate models are developed to approximate IRS pricing within the Monte Carlo CVA pipeline. Each replaces calls to the reference pricer with a single forward pass of a trained network. The models share a common GRU architecture but differ in their training data sources and loss functions, enabling a controlled comparison of the effects of data origin and sensitivity supervision on surrogate accuracy.

8.5.1 Data Generation

Training corpora for the three models are constructed as follows.

Model 1a. Training inputs (a, σ, t, r_t) are sampled from a designed grid spanning a practical parameter range: $a \in [0.01, 0.30]$, $\sigma \in [0.005, 0.030]$, $t \in \{t_k\}_{k=0}^{120}$, and r_t drawn uniformly around the initial short rate. Price labels $V^* = V_{\text{affine}}(t, r_t; a, \sigma)$ are computed from the closed-form formula (8.8). No sensitivity labels are generated.

Model 1b. Training scenarios $(t_k, r_{t_k}^{(i)})$ are extracted directly from the outer Monte Carlo paths used in the CVA computation described in Section 8.4. This in-distribution design ensures the surrogate is trained on the same domain of short-rate realisations it encounters at evaluation time. Price labels $V^* = V_{\text{MC}}(t_k, r_{t_k}^{(i)})$ are computed via the simulation-based pricer (8.13). No sensitivity labels are generated.

Model 2. Training scenarios are the same simulation-drawn pairs as Model 1b. Price labels are likewise from the simulation-based pricer. In addition, key-rate DV01 labels:

$$\text{DV01}_k^* = \frac{\partial V_{\text{MC}}}{\partial y_k}, \quad k = 0, 1, \dots, 9 \quad (8.16)$$

are computed by automatic differentiation through the simulation pricing function, following [55]. These sensitivity labels are used jointly with the price labels in the Sobolev training objective described in Section 8.5.4.

8.5.2 Model Inputs

All three models receive the same input representation. The primary sequence is a vector of zero yields $\{y(0, T_k)\}_{k=1}^{10}$ at ten tenor knot points spanning three months to ten years, providing a discretised snapshot of the prevailing term structure. A scalar

vector (a, σ, t, r_t) supplies the Hull-White parameters and the current state. Yield curve values are used as the sequence input rather than discount factors because the DV01 labels in Model 2 are defined as derivatives with respect to yields; providing yields directly ensures that the gradient $\partial \hat{V} / \partial y_k$ obtained by backpropagation is well-defined and numerically stable across all three models.

8.5.3 Architecture

A single GRU architecture is used for all three models without structural modification between them. The network processes the ten-element yield sequence through L stacked GRU layers, whose update, reset, candidate, and hidden-state equations are given in (7.7)–(7.10) of the Theoretical Foundations. The final hidden state $\mathbf{h}_{10} \in \mathbb{R}^H$ is concatenated with the scalar input vector (a, σ, t, r_t) and passed through a linear output layer to produce the predicted swap value $\hat{V}$. Sensitivity outputs $\partial \hat{V} / \partial y_k$ are obtained by backpropagation through the network at inference time, requiring no architectural modification. The same network can therefore be trained under any of the three loss functions in Section 8.5.4 without alteration. A depth of $L = 2$ layers and hidden dimension $H = 64$ are used as the base configuration, reflecting the low intrinsic dimensionality of the pricing function under the one-factor Hull-White model; these are verified by grid search on the validation set.

8.5.4 Training Objectives

The three models are distinguished solely by their loss functions.

Model 1a minimises the mean squared error on the analytical price labels:

$$\mathcal{L}_{1a} = \frac{1}{M} \sum_{m=1}^M (\hat{V}_m - V_{\text{affine},m}^*)^2 \quad (8.17)$$

Model 1b minimises the mean squared error on the simulation price labels:

$$\mathcal{L}_{1b} = \frac{1}{M} \sum_{m=1}^M (\hat{V}_m - V_{\text{MC},m}^*)^2 \quad (8.18)$$

Model 2 minimises a Sobolev loss that penalises errors in both price and sensitivities jointly [55]:

$$\mathcal{L}_2 = \frac{1}{M} \sum_{m=1}^M \left[(\hat{V}_m - V_{\text{MC},m}^*)^2 + \lambda \sum_{k=0}^9 \left(\frac{\partial \hat{V}_m}{\partial y_k} - \text{DV01}_{k,m}^* \right)^2 \right] \quad (8.19)$$

The weight $\lambda > 0$ balances the price and sensitivity terms. Its magnitude is informed by the ratio of the typical price error to the typical DV01 error on the validation set;

values in the range $\lambda \in [0.1, 1.0]$ are evaluated by grid search. All three models are optimised using the Adam algorithm [60] with an initial learning rate of 10^{-3} and a cosine annealing schedule. Training, validation, and test sets are constructed from disjoint subsets of the generated data; early stopping on validation loss selects the final checkpoint.

8.6 xVA CALCULATION

The surrogate models are designed for a specific purpose: to replace the analytical Hull-White pricer at each simulated scenario in a Monte Carlo-based CVA calculation. CVA measures the market value of counterparty default risk, and computing it requires repricing the swap across the full set of simulated future scenarios. At production scale, this repricing step dominates the computational cost; replacing the analytical pricer with a trained surrogate reduces each scenario valuation to a single forward pass of fixed cost.

Under the assumptions of Section 8.2, the unilateral CVA is:

$$\text{CVA} = (1 - R) \int_0^{T_n} \text{EPE}(t) \lambda e^{-(\lambda + r_0)t} dt \quad (8.20)$$

where $R = 0.40$ is the recovery rate, λ is the constant hazard rate, r_0 is the initial short rate, and $\text{EPE}(t)$ is the expected positive exposure profile from Section 8.4. Discretising over the 120 monthly monitoring dates:

$$\text{CVA} \approx (1 - R) \sum_{k=1}^{120} \text{EPE}(t_k) \lambda e^{-(\lambda + r_0)t_k} \Delta t \quad (8.21)$$

where $\Delta t = 1/12$. In the surrogate-based calculation, the EPE profile is constructed by replacing $V_{\text{IRS}}^{(i)}(t_k, r_{t_k}^{(i)})$ in (8.14) with the surrogate output $\hat{V}^{(i)}(t_k)$. The gap between surrogate-based CVA and analytical CVA is a direct measure of the pricing error that accumulates across the full exposure simulation, and is one of the primary metrics used to compare Model 1a and Model 1b in the Results chapter.

Part IV

Results and Analysis

Part V

Conclusion and Future Work

Part VI

Appendices

Appendix A

Mathematical Proofs and Derivations

Appendix B

Code Implementation Details

Appendix C

Additional Numerical Results

Bibliography

- [1] F. M. Ametrano and M. Bianchetti. *Everything you always wanted to know about multiple interest rate curve bootstrapping but were afraid to ask*, volume 2219548. SSRN, 2013.
- [2] D. P. Anderson and U. Ulrych. Accelerated american option pricing with deep neural networks. *Quantitative Finance and Economics*, 7(1):207–228, 2023. doi: 10.3934/qfe.2023011.
- [3] K. Andersson and C. W. Oosterlee. A deep learning approach for computations of exposure profiles for high-dimensional bermudan options. *Applied Mathematics and Computation*, 408:126332, 2021.
- [4] K. Andersson and C. W. Oosterlee. Deep learning for CVA computations of large portfolios of financial derivatives. *Applied Mathematics and Computation*, 409: 126399, 2021. doi: 10.1016/j.amc.2021.126399.
- [5] E. Avsar. The logic and practice of yield curve construction. *Available at SSRN 2594520*, 2015.
- [6] Y. Bai, T. Chaolu, and S. Bilige. The application of improved physics-informed neural network (IPINN) method in finance. *Nonlinear Dynamics*, 107:3655–3667, 2022. doi: 10.1007/s11071-021-07146-z.
- [7] Bank for International Settlements. Zero-coupon yield curves: Technical documentation. BIS Papers 25, Bank for International Settlements, 2005.
- [8] Basel Committee on Banking Supervision. Basel III: A global regulatory framework for more resilient banks and banking systems. Technical Report bcbs153, Bank for International Settlements, Dec. 2010. URL <https://www.bis.org/publ/bcbs153.pdf>. Revised June 2011.
- [9] C. Beck, M. Hutzenthaler, A. Jentzen, and B. Kuckuck. An overview on deep learning-based approximation methods for partial differential equations. *arXiv preprint arXiv:2012.12348*, 2020.

- [10] S. Becker, A. Jentzen, M. S. Müller, A. Pesce, and P. von Wurstemberger. Learning the random variables in Monte Carlo simulations with stochastic gradient descent: machine learning for parametric PDEs and financial derivative pricing. *Mathematical Finance*, 33(3):605–635, 2023. doi: 10.1111/mafi.12405.
- [11] M. Bianchetti. Two curves, one price: Pricing & hedging interest rate derivatives decoupling forwarding and discounting yield curves. *One Price: Pricing & Hedging Interest Rate Derivatives Decoupling Forwarding and Discounting Yield Curves (November 14, 2008)*, 2008.
- [12] F. Black and M. Scholes. The pricing of options and corporate liabilities. *Journal of political economy*, 81(3):637–654, 1973.
- [13] S. Blyth. *An introduction to quantitative finance*. Oxford University Press, 2014.
- [14] D. Brigo, F. Mercurio, et al. *Interest rate models: theory and practice*, volume 2. Springer, 2001.
- [15] D. Brigo, M. Morini, and A. Pallavicini. *Counterparty credit risk, collateral and funding: with pricing cases for all asset classes*. John Wiley & Sons, 2013.
- [16] C. Burgard and M. Kjaer. In the balance. *C. Burgard, M. Kjaer. In the balance, Risk, November*, pages 72–75, 2011.
- [17] C. Burgard and M. Kjaer. Partial differential equation representations of derivatives with bilateral counterparty risk and funding costs. *The Journal of Credit Risk*, 7: 75–93, 09 2011. doi: 10.21314/JCR.2011.131.
- [18] E. Canabarro and D. Duffie. Measuring and marking counterparty risk. *Asset/Liability Management for Financial Institutions, Institutional Investor Books*, 2003.
- [19] J. Cao, J. Chen, J. Hull, and Z. Poulos. Deep hedging of derivatives using reinforcement learning. *The Journal of Financial Data Science*, 3(1):10–27, 2020. doi: 10.3905/jfds.2020.1.052.
- [20] G. Cesari, J. Aquilina, N. Charpillon, Z. Filipovic, G. Lee, and I. Manda. *Modelling, pricing, and hedging counterparty credit exposure: A technical guide*. Springer Science & Business Media, 2009.
- [21] H. Chang. Analysis of stochastic volatility models in financial derivatives pricing. *Financial Engineering and Risk Management*, 7(6):74–82, 2024. doi: 10.23977/ferm.2024.070611.

- [22] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio. Learning phrase representations using RNN encoder-decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, pages 1724–1734, 2014.
- [23] M. Choi and H.-G. Kang. How to calibrate Gaussian two-factor model using swaption. *PLoS ONE*, 18(1):e0280829, 2023. doi: 10.1371/journal.pone.0280829.
- [24] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio. Empirical evaluation of gated recurrent neural networks on sequence modeling. *arXiv preprint arXiv:1412.3555*, 2014.
- [25] J. C. Cox and S. A. Ross. The valuation of options for alternative stochastic processes. *Journal of financial economics*, 3(1-2):145–166, 1976.
- [26] C. Cuchiero, W. Khosrawi, and J. Teichmann. A generative adversarial network approach to calibration of local stochastic volatility models. *Risks*, 8(4):101, 2020. doi: 10.3390/risks8040101.
- [27] Y. Cui, L. Li, G. Zhang, and W. Zhang. Learning pricing maps for financial derivatives with deep operator networks. *Available at SSRN 4670716*, 2023.
- [28] G. Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2(4):303–314, 1989.
- [29] J. De Spiegeleer, D. B. Madan, S. Reyners, and W. Schoutens. Machine learning for quantitative finance: fast derivative pricing, hedging and fitting. *Quantitative Finance*, 18(10):1635–1643, 2018. doi: 10.1080/14697688.2018.1495335.
- [30] G. Deelstra, L. A. Grzelak, and F. L. Wolf. Accelerated computations of sensitivities for xva. *International Journal of Computer Mathematics*, 101(8):842–864, 2024.
- [31] V. L. Dhandapani and S. Jain. Optimizing neural networks for Bermudan option pricing: Convergence acceleration, future exposure evaluation and interpolation in counterparty credit risk. *arXiv preprint*, 2024. doi: 10.48550/arxiv.2402.15936.
- [32] F. X. Diebold and C. Li. Forecasting the term structure of government bond yields. *Journal of Econometrics*, 130(2):337–364, 2006. doi: 10.1016/j.jeconom.2005.03.005.
- [33] D. J. Duffy and A. Germani. *C# for Financial Markets*, volume 405. John Wiley & Sons, 2013.

- [34] D. Elbrächter, P. Grohs, A. Jentzen, and C. Schwab. DNN expression rate analysis of high-dimensional PDEs: Application to option pricing. *Constructive Approximation*, 55:3–71, 2021. doi: 10.1007/s00365-021-09541-6.
- [35] I. Frodé and V. Samberg. Neural networks for credit risk and xva in a front office pricing environment. *Master's Theses in Mathematical Sciences*, 2022.
- [36] X. Glorot, A. Bordes, and Y. Bengio. Deep sparse rectifier neural networks. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, pages 315–323. JMLR Workshop and Conference Proceedings, 2011.
- [37] A. Gnoatto, A. Picarelli, and C. Reisinger. Deep xva solver – a neural network based counterparty credit risk management framework, 2022.
- [38] I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. MIT Press, 2016. <http://www.deeplearningbook.org>.
- [39] I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. MIT Press, Cambridge, Massachusetts, 2016.
- [40] L. Goudenège, A. Molent, and A. Zanette. Computing XVA for American basket derivatives by machine learning techniques. *Computational Management Science*, 2025. doi: 10.1007/s10287-025-00540-7.
- [41] A. Green. *XVA: Credit, funding and Capital Valuation adjustments*. Wiley, 2016.
- [42] J. Gregory. *The XVA Challenge: Counterparty credit risk, funding, collateral, and capital*. Wiley, 2015.
- [43] A. Gupta and M. G. Subrahmanyam. Pricing and hedging interest rate options: Evidence from cap–floor markets. *Journal of Banking & Finance*, 29(3):701–733, 2005.
- [44] R. S. Gürkaynak, B. Sack, and J. H. Wright. The U.S. Treasury yield curve: 1961 to the present. *Journal of Monetary Economics*, 54(8):2291–2304, 2007. doi: 10.1016/j.jmoneco.2007.06.029.
- [45] P. S. Hagan and G. West. Methods for constructing a yield curve. *Wilmott Magazine*, May, pages 70–81, 2008.
- [46] J. Han, A. Jentzen, and W. E. Solving high-dimensional partial differential equations using deep learning. *Proceedings of the National Academy of Sciences*, 115(34):8505–8510, 2018.

- [47] J. M. Harrison and D. M. Kreps. Martingales and arbitrage in multiperiod securities markets. *Journal of Economic Theory*, 20(3):381–408, 1979. ISSN 0022-0531. doi: 10.1016/0022-0531(79900437). URL <https://www.sciencedirect.com/science/article/pii/0022053179900437>.
- [48] J. M. Harrison and S. R. Pliska. Martingales and stochastic integrals in the theory of continuous trading. *Stochastic Processes and their Applications*, 11(3):215–260, 1981.
- [49] T. Hastie, R. Tibshirani, and J. Friedman. *The Elements of Statistical Learning*. Springer, New York, 2 edition, 2009.
- [50] A. Hirsa and S. N. Neftci. *An introduction to the mathematics of financial derivatives*. Academic press, 2013.
- [51] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- [52] K. Hornik. Approximation capabilities of multilayer feedforward networks. *Neural Networks*, 4(2):251–257, 1991.
- [53] K. Hornik, M. Stinchcombe, and H. White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366, 1989.
- [54] B. Horvath, A. Muguruza, and M. Tomas. Deep learning volatility: a deep neural network perspective on pricing and calibration in (rough) volatility models. *Quantitative Finance*, 21(1):11–27, 2021. doi: 10.1080/14697688.2020.1817974.
- [55] B. Huge and A. Savine. Differential machine learning. *arXiv preprint arXiv:2005.02347*, 2020.
- [56] J. Hull. *Risk management and Financial Institutions*. Wiley, 2018.
- [57] J. C. Hull. *Options, Futures, and Other Derivatives*. Pearson, Upper Saddle River, NJ, 11th edition, 2022. ISBN 978-0136939979.
- [58] J. M. Hutchinson, A. W. Lo, and T. Poggio. A nonparametric approach to pricing and hedging derivative securities via learning networks. *The journal of Finance*, 49(3):851–889, 1994.
- [59] M. Hutzenthaler, A. Jentzen, and P. von Wurstemberger. Overcoming the curse of dimensionality in the approximative pricing of financial derivatives with default risks. *Electronic Journal of Probability*, 25:1–73, 2020. doi: 10.1214/20-ejp423.

- [60] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. In *Proceedings of the Third International Conference on Learning Representations*, 2015.
- [61] KPMG in Ireland. Navigating the complexities of derivative valuation. URL <https://kpmg.com/ie/en/insights/asset-management/navigating-the-complexities-of-derivative-valuation.html>. Asset Management – Our Insights.
- [62] Y.-K. Kwok. *Mathematical models of financial derivatives*. Springer, 2008.
- [63] S. Lee, J. Huh, and S. Jeong. Deeponet-based surrogate modeling for bond option pricing. *AIMS Mathematics*, 11(3):5853, 2026.
- [64] A. Menegon and A. Carapelli. Real-time exposures and xvas: a neural network approach. 2023.
- [65] M. Mrad and N. Iassamen. Differential deep learning and physics-informed neural networks for fast derivative pricing. 2025.
- [66] N. Nisha, N. Puri, N. Rajput, and N. Bhardwaj. Foundations and trends in option pricing models: a 45 years global examination based on bibliometric analysis. *Qualitative Research in Financial Markets*, 2024. doi: 10.1108/qrfm-05-2022-0092.
- [67] C. W. Oosterlee and L. A. Grzelak. *Mathematical modeling and computation in finance: with exercises and Python and MATLAB computer codes*. World Scientific, 2019.
- [68] J. Ruf and W. Wang. Neural networks for option pricing and hedging: a literature review. *Journal of Computational Finance*, 24(1):1–46, 2020.
- [69] D. E. Rumelhart, G. E. Hinton, and R. J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.
- [70] S. E. Shreve et al. *Stochastic calculus for finance II: Continuous-time models*, volume 11. Springer, 2004.
- [71] L. E. O. Svensson. Estimating and interpreting forward interest rates: Sweden 1992–1994. Working Paper 4871, National Bureau of Economic Research, 1994.
- [72] O. Tsuchiya. *A practical approach to XVA: The evolution of derivatives valuation after the financial crisis*. World Scientific, 2019.

- [73] T. Vanhala. Data-driven calculation of xVA exposure amounts for an interest rate swap portfolio. Master's thesis, Aalto University, 2023. URL <https://aaltodoc.aalto.fi/handle/123456789/120932>.
- [74] S. Wang, H. Wang, and P. Perdikaris. Learning the solution operator of parametric partial differential equations with physics-informed deepnets. *Science advances*, 7(40):eabi8605, 2021.
- [75] E. Weinan. Machine learning and computational mathematics. *arXiv preprint arXiv:2009.14596*, 2020.
- [76] Y. Yu, N. Ganesan, and B. Hientzsch. Backward deep BSDE methods and applications to nonlinear problems. *Risks*, 11(3):61, 2023. doi: 10.3390/risks11030061.
- [77] X. Zhou and V. Abramov. A practical guide to interest rate curve building validations (w/excel replica of bloomberg libor@ github). *Available at SSRN 3329563*, 2019.
- [78] S. Zhu, J. Chan, and D. Bright. Applying machine learning for troubleshooting credit exposure and xva profiles. *Available at SSRN 3404863*, 2019.
- [79] S. H. Zhu and M. Pykhtin. A guide to modeling counterparty credit risk. *GARP Risk Review*, July/August, 2007.